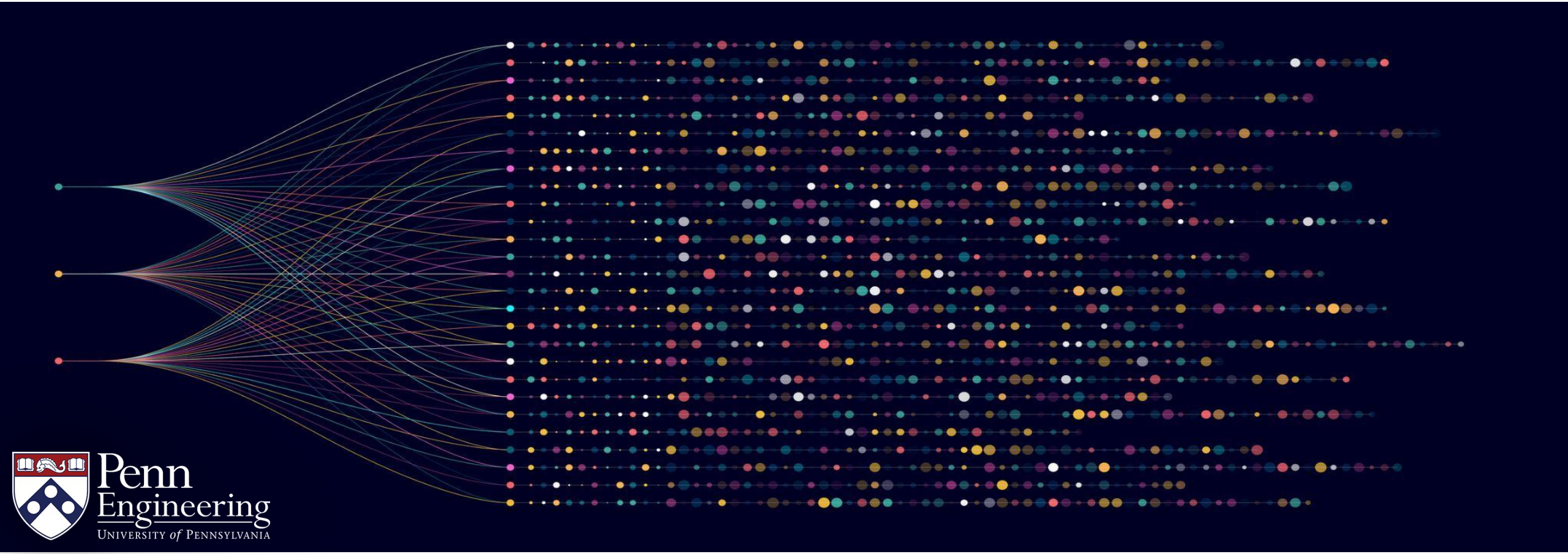
Algorithms for Big Data

Anindya De Erik Waingarten

Estimating Averages: Problem Setup



Penn Engineering
University of Pennsylvania

Objectives: Computing Large Averages

- Motivating Scenario
- Defining the computational problem
- A “brute force” algorithmic benchmark

Motivating Scenario

- You are a software engineer working for a social network
- There is a (very large) database which holds all of user information and maintains usage information
- You are tasked with determining certain aggregate statistics on users and their usage information

Motivating Scenario - Example Questions

- How many users are in Europe?
- How many users posted in the past week?
- How long (on average) did users spend on the platform today?

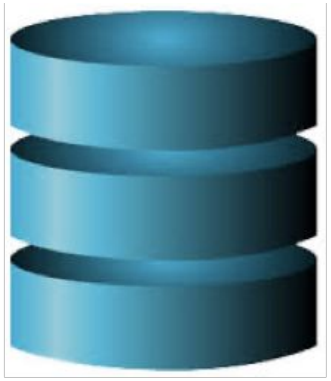
Commonalities: Ask for large averages over the entire dataset

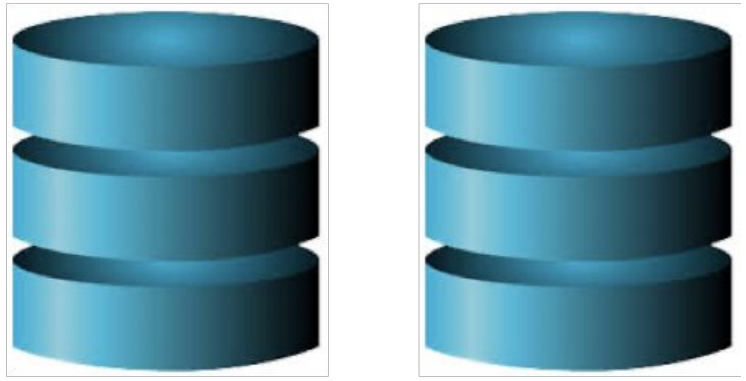
Outline for what is coming:

- First, provide background on the scenario
  - How data is stored
  - How data is accessed
  - What we want to do

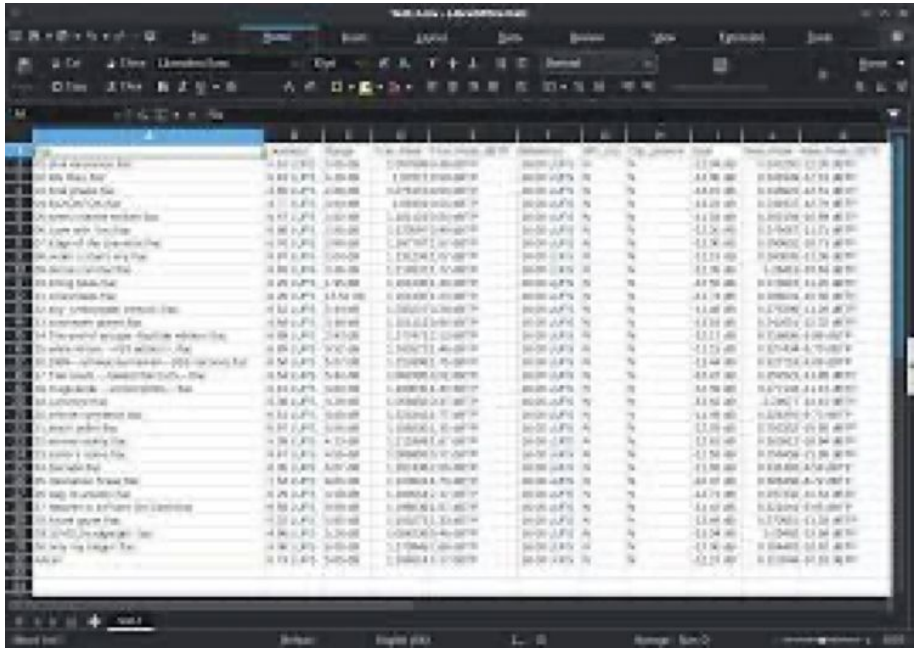
- Then, define computational problem
  - Incorporating elements from the scenario
  - Establish algorithmic benchmarks

Background: What is the data?





Input data
(possibly in the cloud)



Rows may be users/events,
columns are attributes

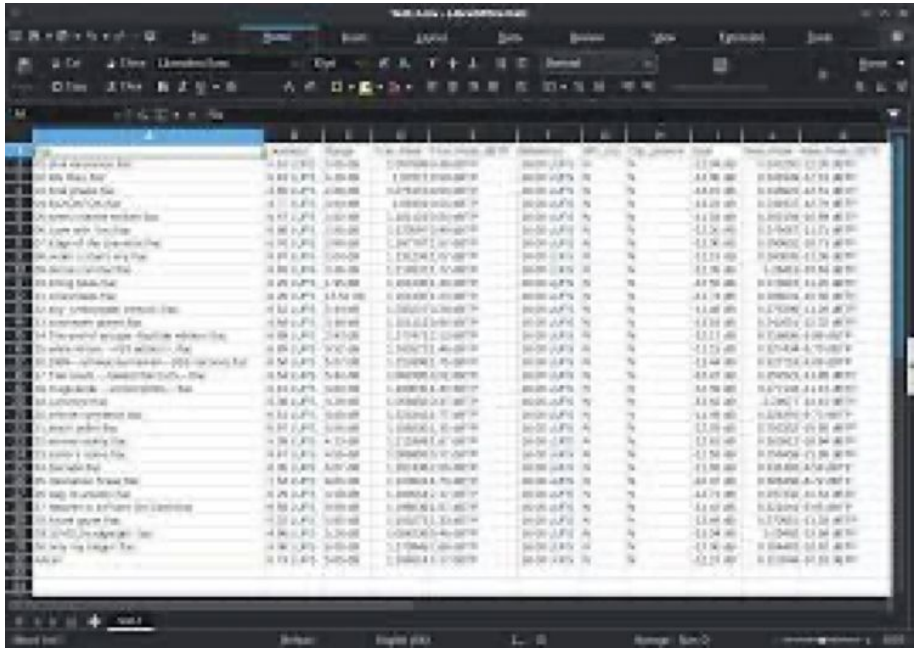
Background: What is the data?

Establish notation:

$$n = \text{ number of items.}$$

$$X_1, \ldots, X_n \in \mathbb{R}$$

$$S = \frac{1}{n} \sum_{i=1}^{n} X_i$$

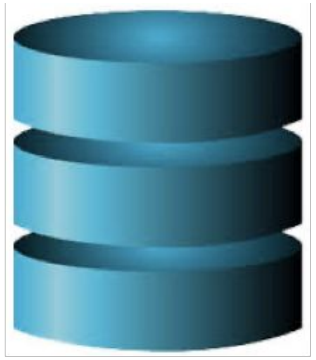


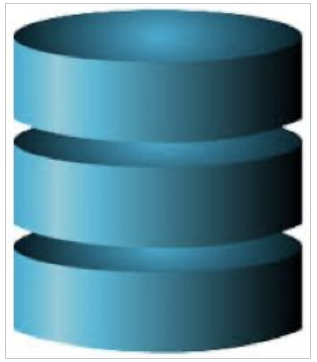
Rows may be users/events, columns are attributes

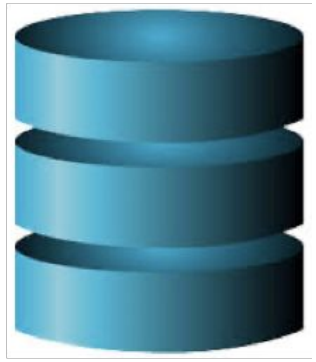
Background: What is the access?

Query

Result of Query







Background: What is the access?

- The input data may not be readily accessible.
  - If data lies in a data center, communication costs may be bottleneck

- Input data is accessible. We want our algorithms to be very fast.
  - The running time of the algorithm is dominated by the number of queries made

Defining the Computational Problem.

Input: $X_1, \ldots, X_n \in \mathbb{R}$

Goal: $S = \frac{1}{n} \sum_{i=1}^{n} X_i$

Queries: X_i

Examples of estimating large averages:

1. How many users are in Europe?
2. How many users posted in the past week?
3. How long did users spend on the platform on average?

Algorithmic Benchmark

Approach: brute force

Algorithm:

- Instantiate a variable which will maintain the sum
- Query every value and maintain sum
- Output the sum divided by n

Questions:

- Is it correct?
- How long should we expect it to take?

Can one be faster?

- Brute force: exact and deterministic
  - downside: slow

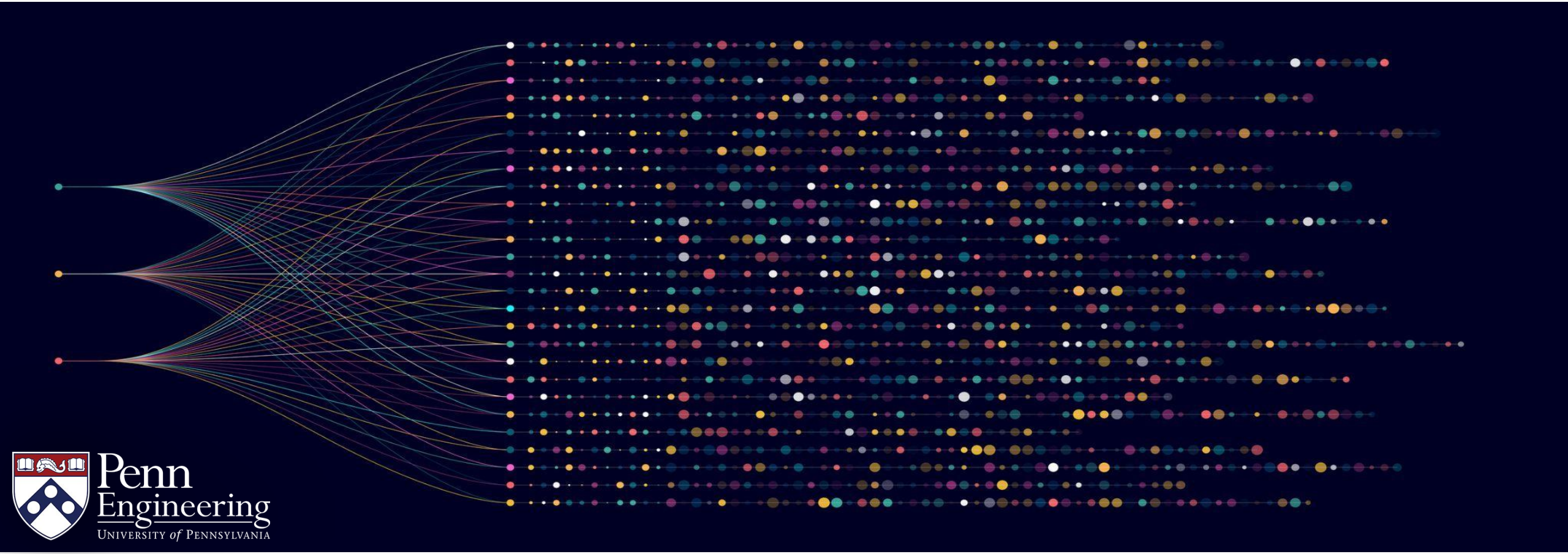
- Do there exist faster algorithms?
  - No, if we insist on exact or deterministic algorithms

- If we allow *approximation* and *randomization*, we can design significantly faster algorithms

Algorithms for Big Data

Anindya De Erik Waingarten

Additive Approximation Guarantees



Penn Engineering
University of Pennsylvania

Objectives: Computing Large Averages

- Estimating large averages

- Additive approximations

Defining the Computational Problem

Input: $X_1, \ldots, X_n \in \mathbb{R}$

Goal: $S = \frac{1}{n} \sum_{i=1}^{n} X_i$

Queries: X_i

Examples of large averages:

1. How many users are in Europe?
2. How many users posted in the past week?
3. How long did users spend on the platform on average?

When is approximation adequate?

- Would an approximation to the output be sufficient?

- How would the result of the algorithm impact downstream decisions (or other algorithms)?

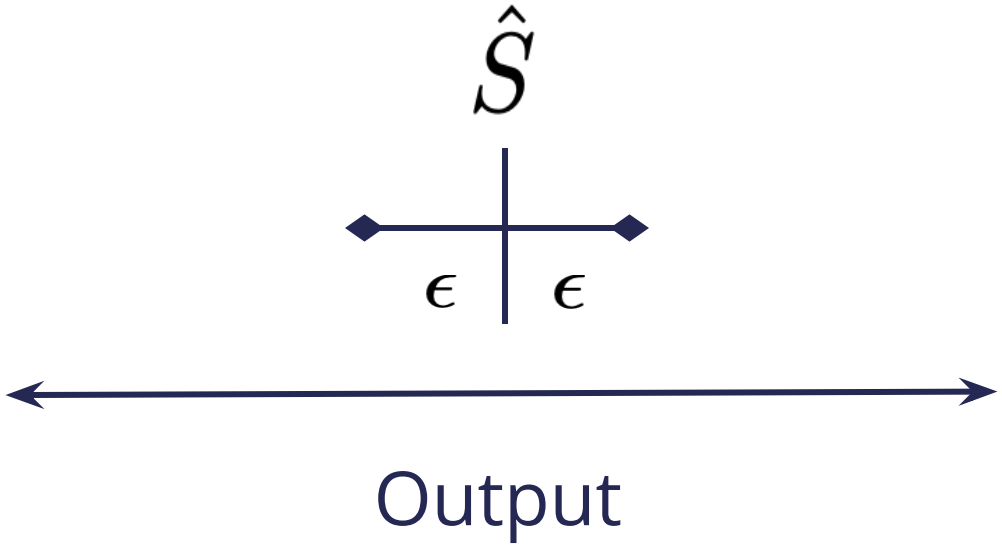
- If input is too large, no choice but to approximate. It is important to know what you are getting.

What do approximation guarantees look like?

- A number approximates another number when they are close

- Execute an algorithm which outputs $\hat{S} \in \mathbb{R}$ such that for a known $\epsilon \geq 0$:

$$|\hat{S} - S| \leq \epsilon$$



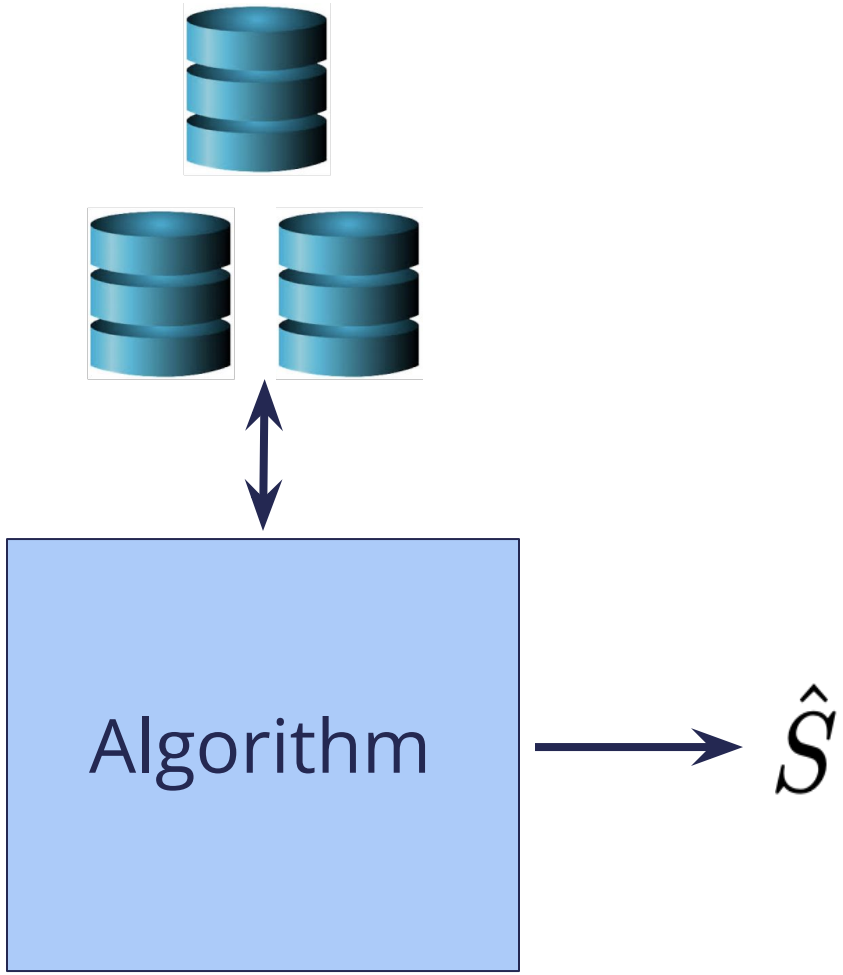
Additive ϵ-approximations

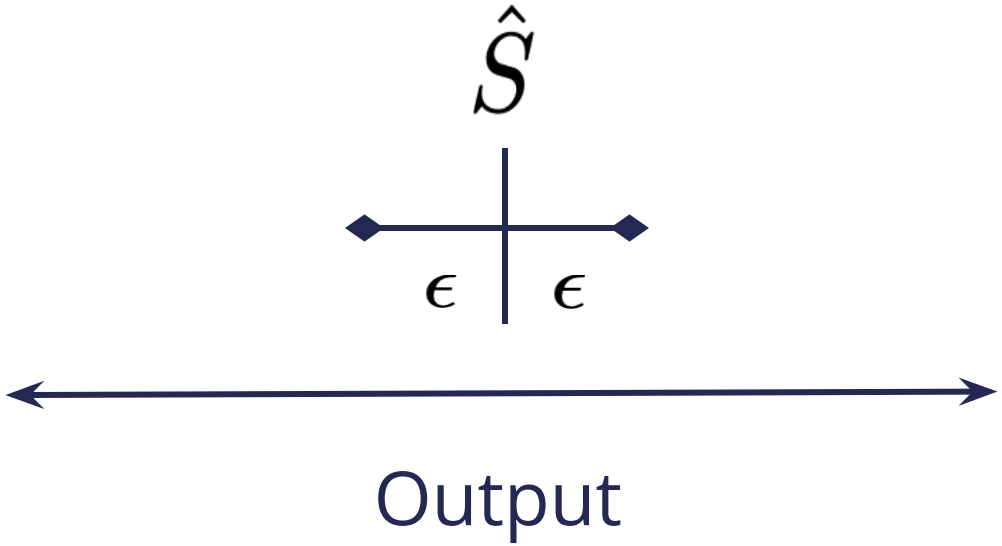
Definition: An algorithm for computing a large average is an additive ϵ-approximation if it satisfies the following guarantees:

- Given input query access to $X_1, \ldots, X_n \in \mathbb{R}$
- It produces an output number $\hat{S}$ which satisfies

$$\left| \hat{S} - \frac{1}{n} \sum_{i=1}^{n} X_i \right| \leq \epsilon.$$

What do approximation guarantees look like?





Interpreting results of approximations

- How many users are in Europe?
  - If accuracy is 0.05, and output is 0.3, then between 25% and 35% of users are in Europe
- How many users posted in the past week?
  - If accuracy is 0.1 and output is 0.9, then between 80% and 100% of users posted in the past week
- How long (on average) did users spend on the platform today?
  - If accuracy is 30 and output is 120, then between 90 and 150 seconds

Approximation guarantees

- Accuracy parameter $\epsilon \geq 0$. As ϵ becomes smaller, the algorithm becomes more accurate. When zero, algorithm is exact.

- As the algorithm becomes more accurate, the number of queries grows. More accurate algorithms are more expensive.

- Big sacrifice. Algorithm will no longer be exactly correct.

Key algorithmic questions:

- Do there exist (more) efficient algorithms when allowing approximation?
  - By more efficient, fewer than $O(n)$ queries, which was the brute force benchmark.

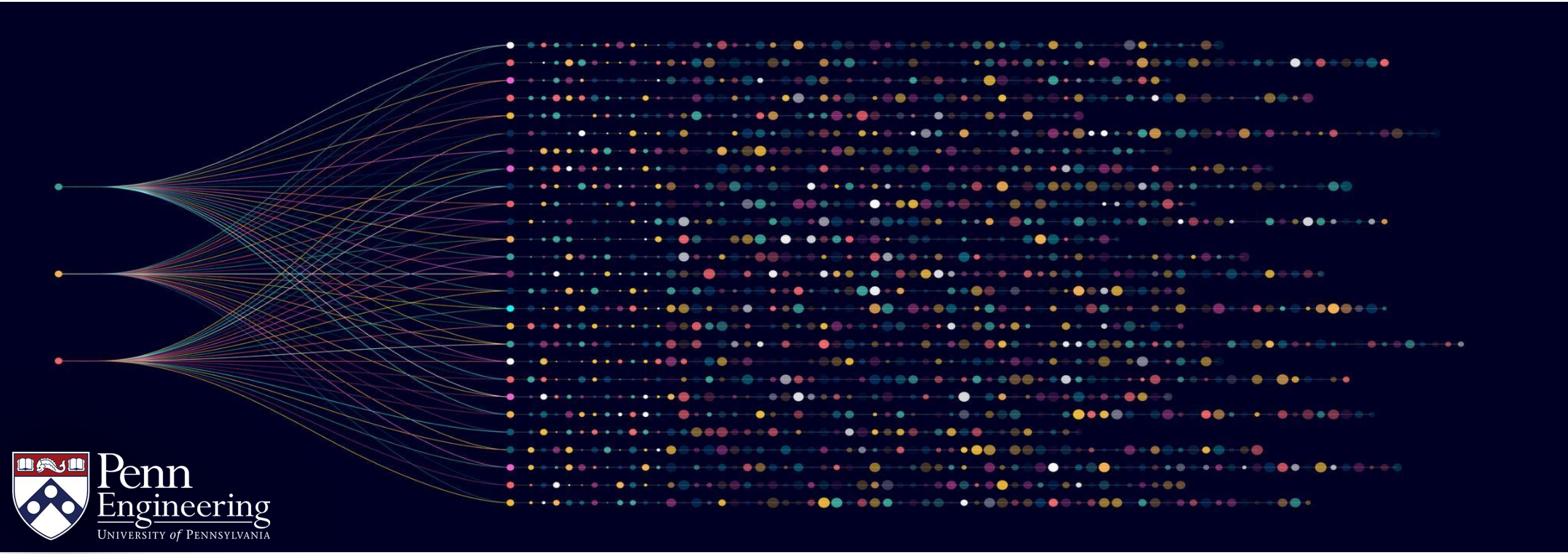
- If so, what is the tradeoff between the number of queries and the accuracy parameter.

- Turns out no, unless algorithm is also randomized.

Algorithms for Big Data

Anindya De Erik Waingarten

Pitfall of Deterministic Algorithms



Penn Engineering
University of Pennsylvania

Objectives: Computing Large Averages

- Randomized algorithms for large averages

- Why deterministic algorithms cannot produce accurate approximations

Defining the Computational Problem.

Input: $X_1, \ldots, X_n \in \mathbb{R}$ and $\epsilon \geq 0$

Goal: Output $\hat{S}$ which satisfies $\left| \hat{S} - \frac{1}{n} \sum_{i=1}^{n} X_i \right| \leq \epsilon.$

Bad news: Deterministic algorithm must make linearly many queries

What is randomization?

- Randomized algorithm output random variables
  - Algorithm “flips coins” internally, specific output is inherently unpredictable

- For approximate guarantees with sublinearly many queries, randomness is unavoidable

Why is randomness necessary?

Input variables:

0	0	1	1	1	1	0	1	0	1	1	0	0	0	1	1

A deterministic algorithm (which does not use any randomness) is a rule for selecting queries

Goal: two inputs with different averages which look the same to a deterministic algorithm

We construct two examples to “fool” algorithm

Input variables:

Assume a deterministic algorithm making 5 queries.

- When query position i, make that 0

Possible Inputs:

	Average
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0
0 0 1 1 0 1 1 1 1 0 1 1 1 1 1 0	11/16

Precise bounds for the “fooling set” argument

- Fix a value of n and a deterministic algorithm making q queries
- Two inputs:
  - All 0’s input — Average = 0
  - The q queries are 0, rest are 1’s — Average = 1 - q/n
- Best case scenario: $\frac{1}{2}(1 - q/n) \leq \epsilon$.
- Conclusion: queries must be at least $n(1 - 2\epsilon)$

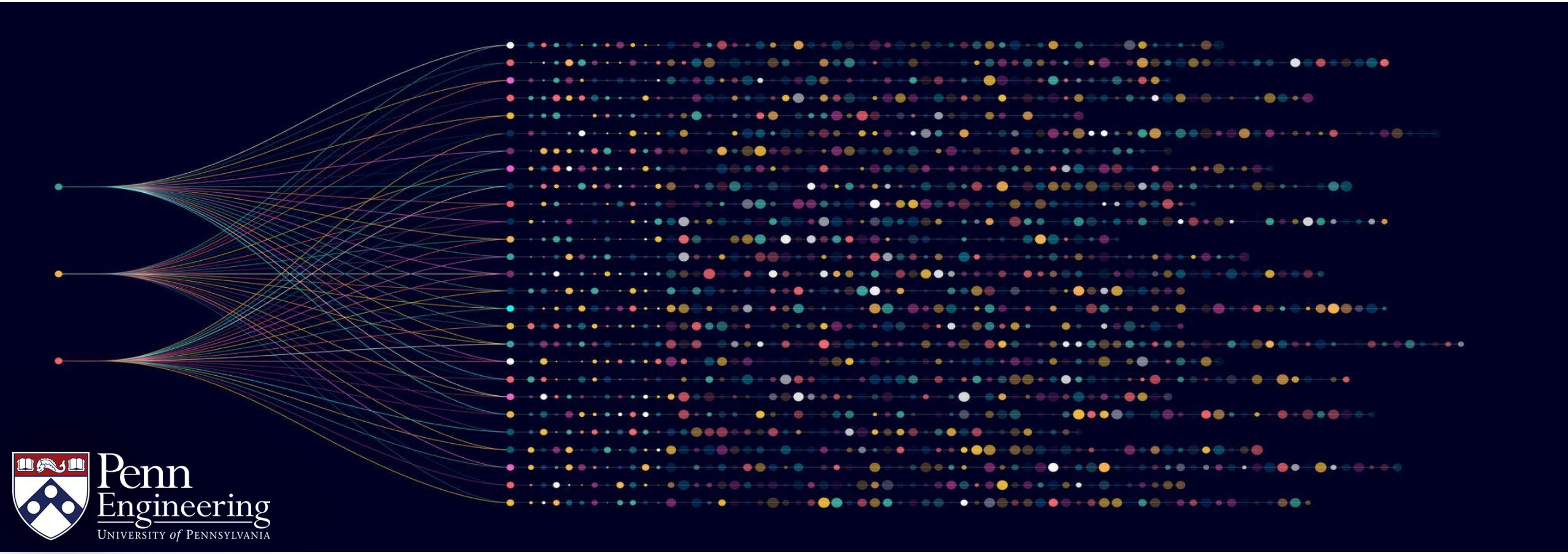
The pitfall of deterministic algorithms

- Deterministic algorithms commit to a fixed action
- If the number of queries is small, there are many values which were not observed
- There are two different inputs which are indistinguishable to the algorithm, but have significantly different answers

Algorithms for Big Data

Anindya De Erik Waingarten

Randomization: Pros and Cons



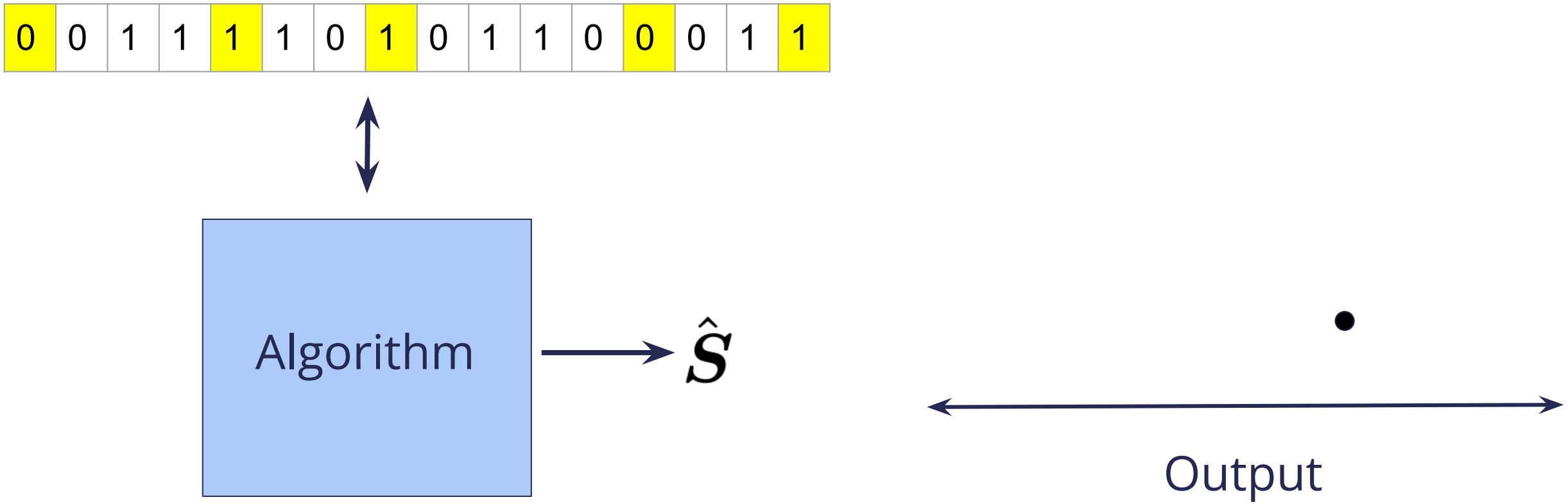
Penn Engineering
University of Pennsylvania

Objectives: Computing Large Averages

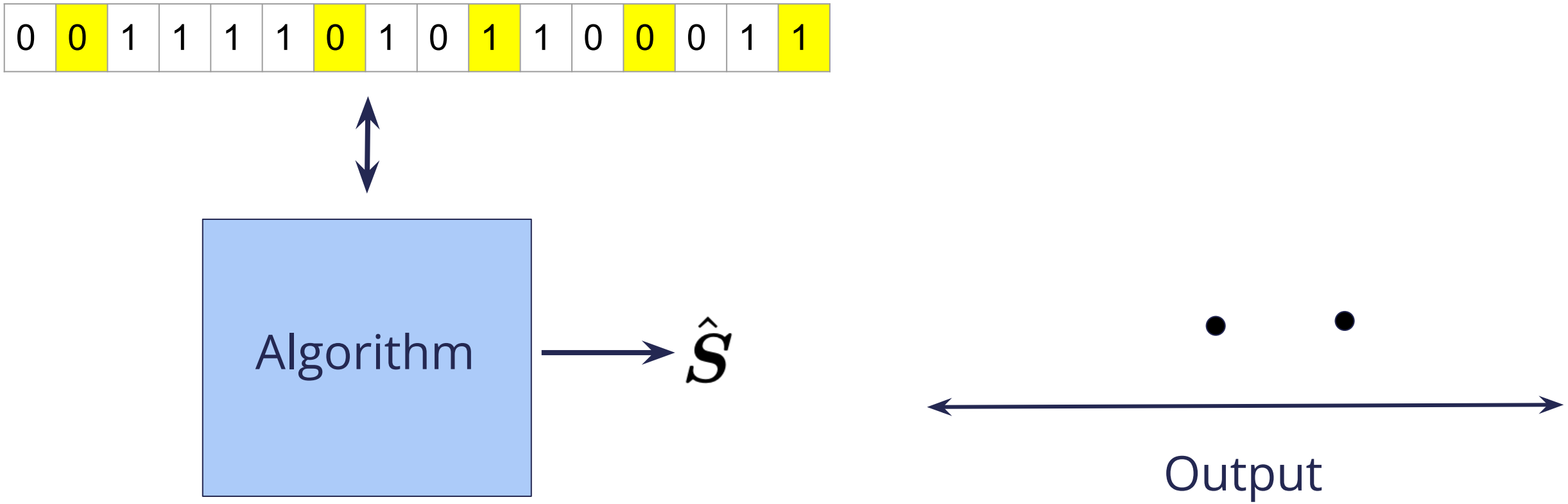
- Randomized algorithms

- Pros and cons of randomization

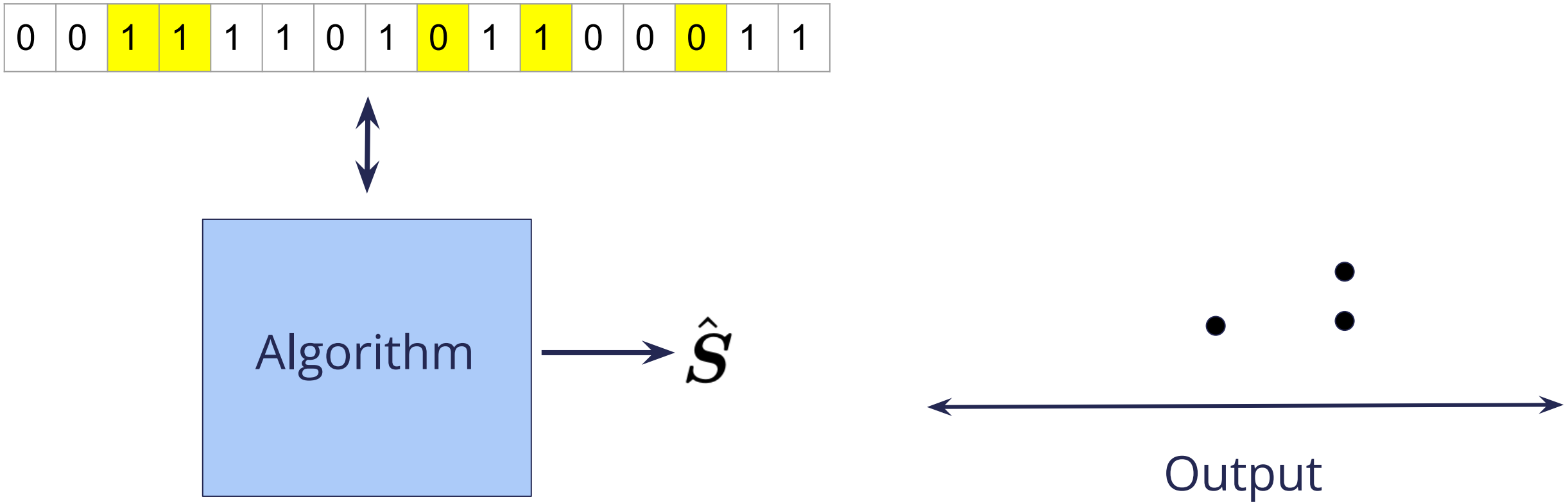
What if the algorithm does not commit to certain queries?



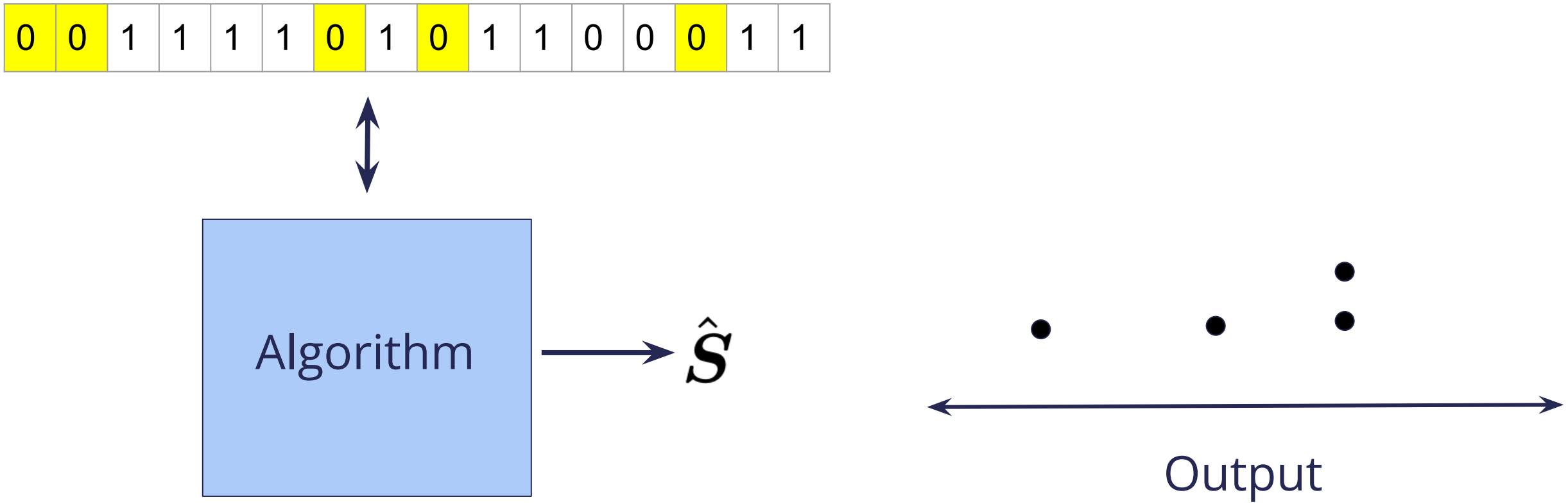
What if the algorithm does not commit to certain queries?



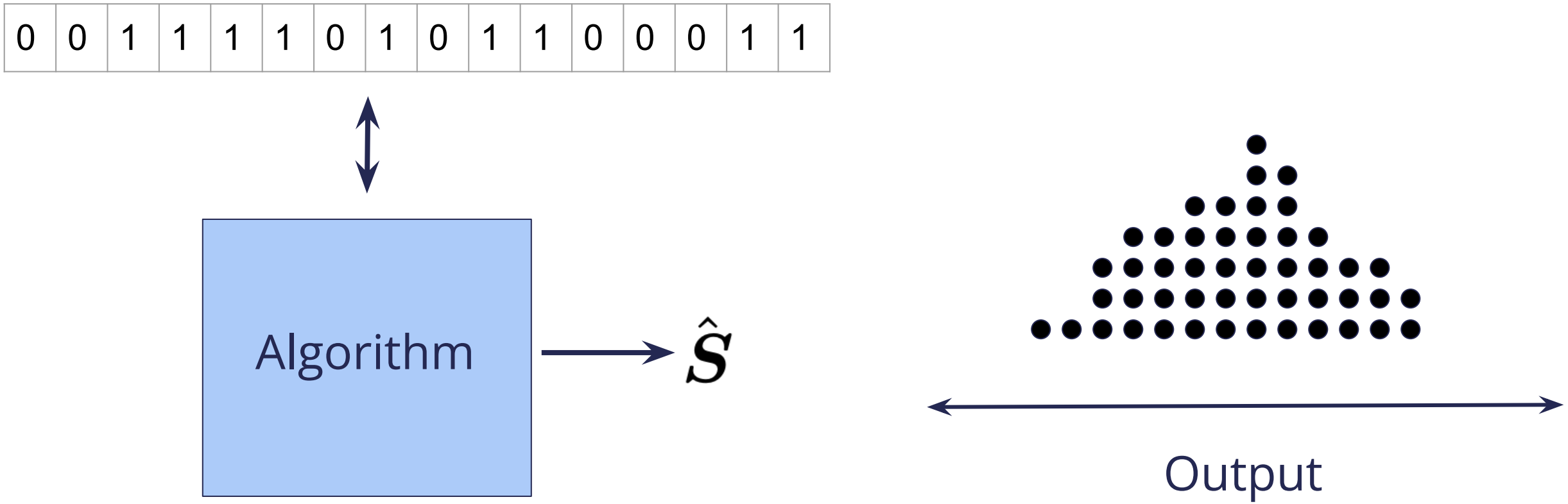
What if the algorithm does not commit to certain queries?



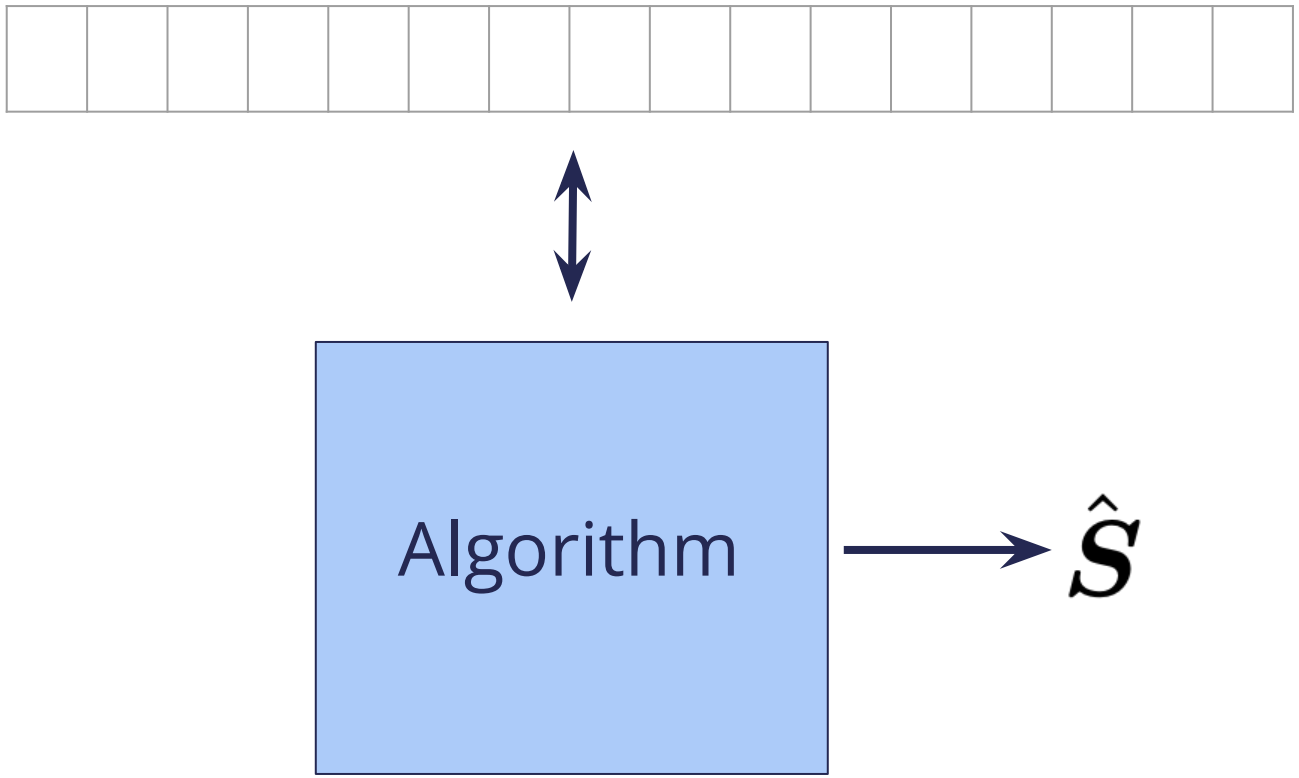
What if the algorithm does not commit to certain queries?



What if the algorithm does not commit to certain queries?



Visualizing a randomized algorithm



- Output of the algorithm is a random variable
- Correctness in terms of properties of the random variable

Pros and Cons of Randomized Algorithms

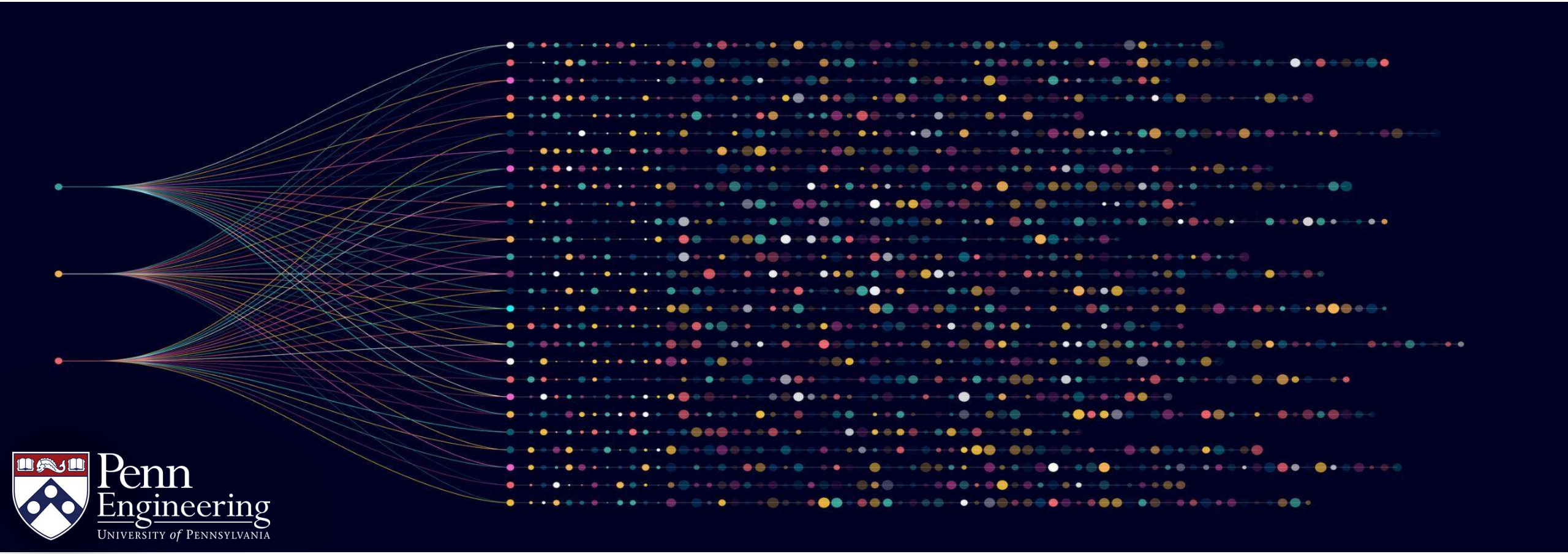
- Pros:
  - Algorithm doesn't commit to fixed set of queries
  - Cannot construct indistinguishable instances, because we don't know queries

- Cons:
  - Every time we execute algorithm, may get a different answer
  - Hard to debug
  - Algorithm's output is a random variable, must reason probabilistically about guarantees

Algorithms for Big Data

Anindya De Erik Waingarten

Probabilistic Guarantees



Penn Engineering
University of Pennsylvania

Objectives: Computing Large Averages

- Probabilistic guarantees for algorithms

- Additive approximations with high probability

Guarantees of Randomized Algorithms

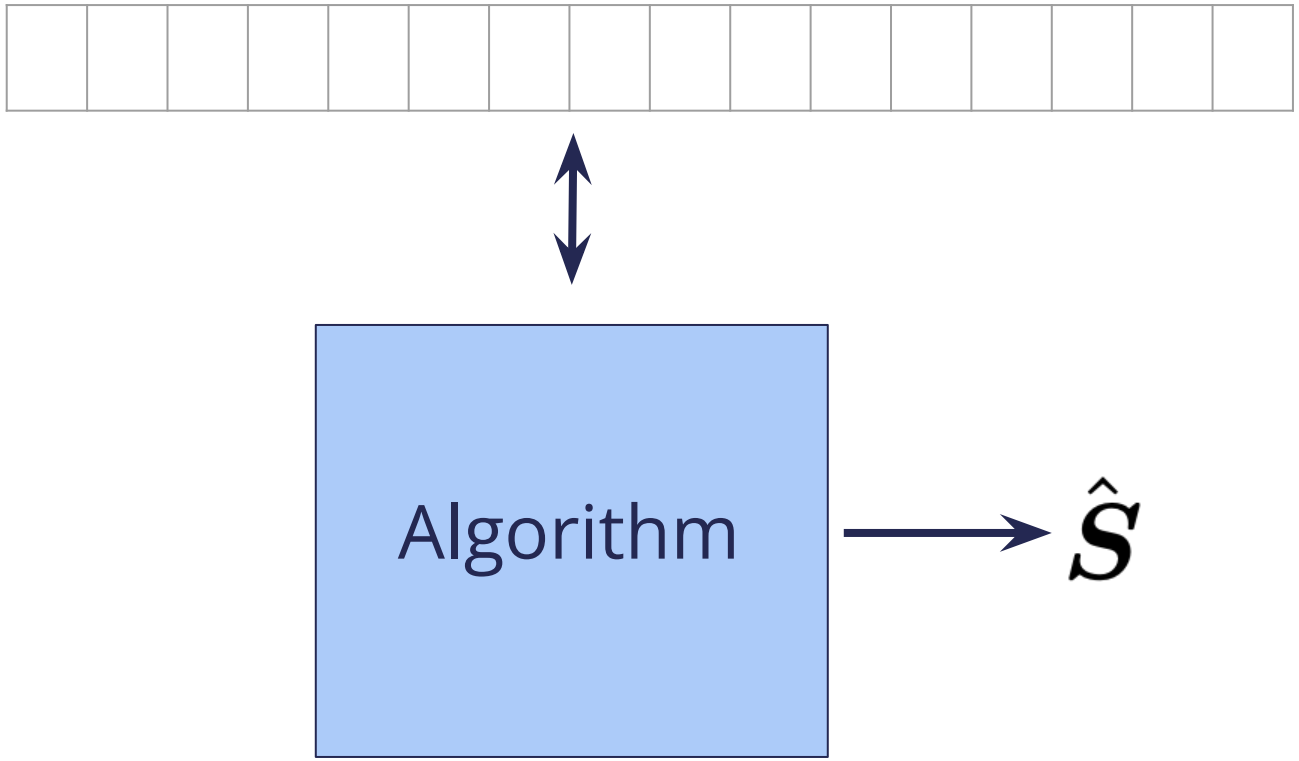
Definition: A randomized algorithm for approximating an average is an algorithm with the following guarantees:

- Input:
  - Query access to $X_1, \ldots, X_n \in \mathbb{R}$
  - Accuracy parameter ϵ
  - Source of randomness
  - Failure probability $\delta \in (0, 1)$
- Guarantee: $$\Pr\left[\left|\hat{\boldsymbol{S}} - \frac{1}{n}\sum_{i=1}^{n} X_i\right| \leq \epsilon\right] \geq 1 - \delta.$$

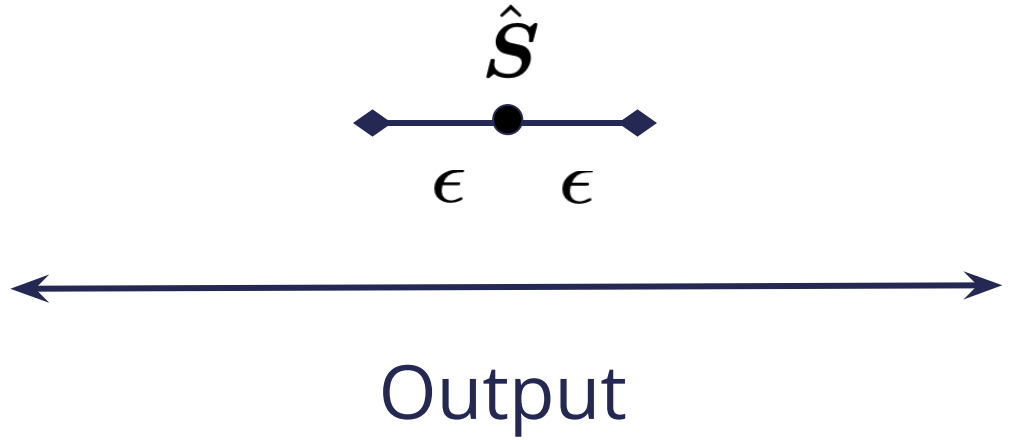
Interpreting results of randomized approximations

- How many users are in Europe?
  - Suppose accuracy is 0.05 and failure probability is 0.01
  - Output of algorithm is 0.3
  - There is a 1% chance algorithm is incorrect. If correct, then between 25% and 35% of users are in Europe.
- How many users posted in the past week?
  - Suppose accuracy is 0.1 and failure probability is 0.05
  - Output of algorithm is 0.9
  - There is a 5% chance algorithm is incorrect. If correct, then between 80% and 100% of users posted in the past week.
- How long (on average) did users spend on the platform today?
  - Suppose accuracy is 30 and failure probability 0.0001
  - Output of algorithm is 120
  - There is a 0.01% chance algorithm is incorrect. If correct, then users spent between 90 and 150 seconds on the platform on average.

Approximate and randomized guarantees: **user perspective**

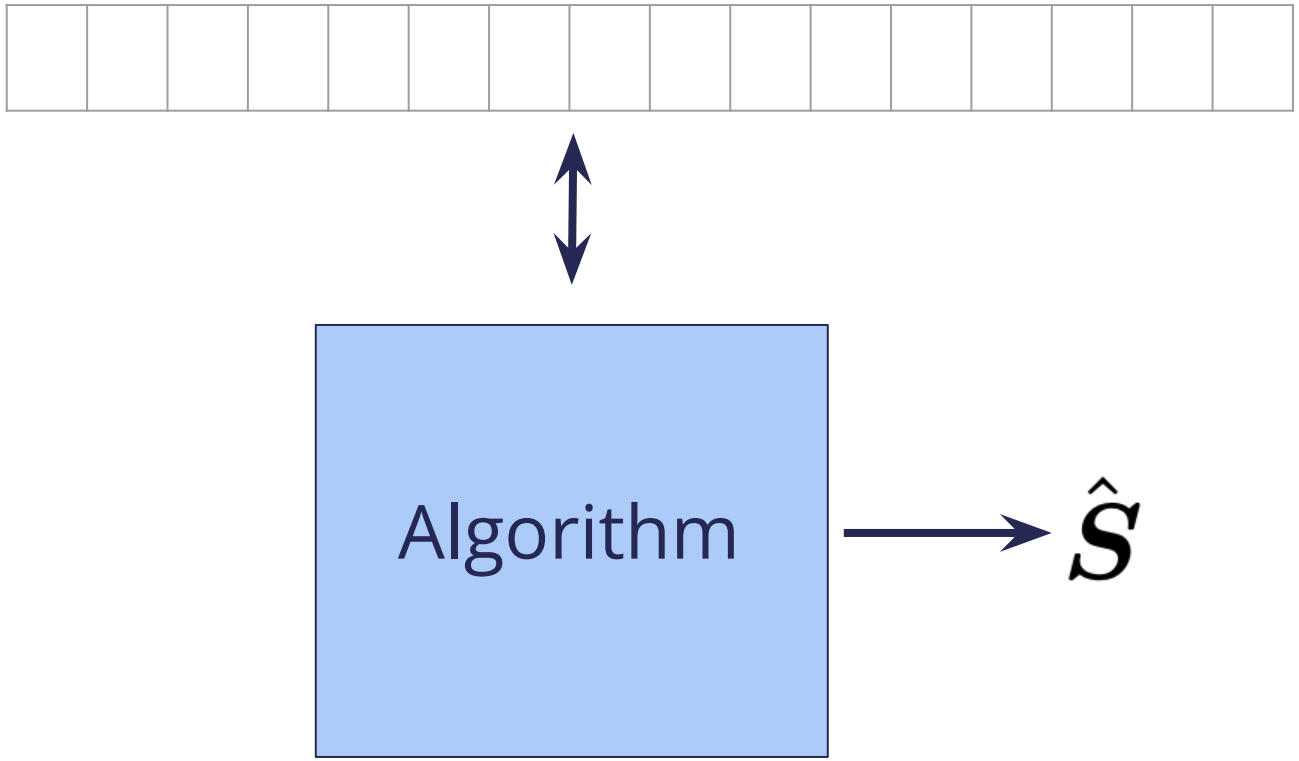


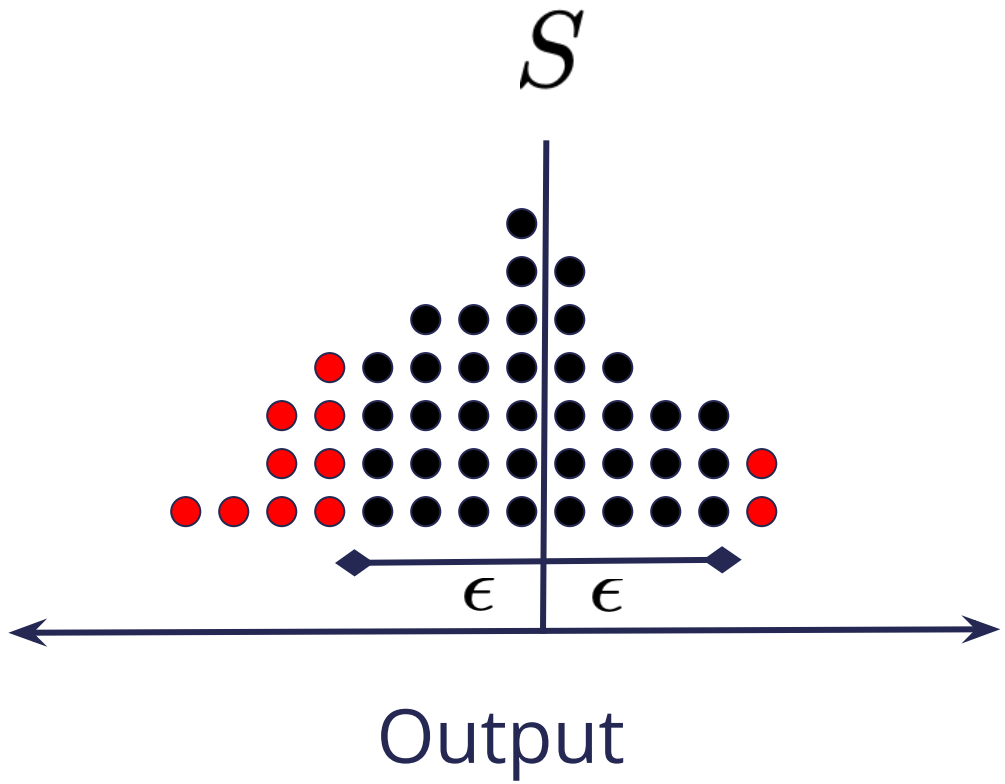
may be wrong with probability δ



ϵ : accuracy δ: failure probability

Approximate and randomized guarantees: **algorithm perspective**





Output

ϵ : accuracy δ: failure probability

Key algorithmic questions:

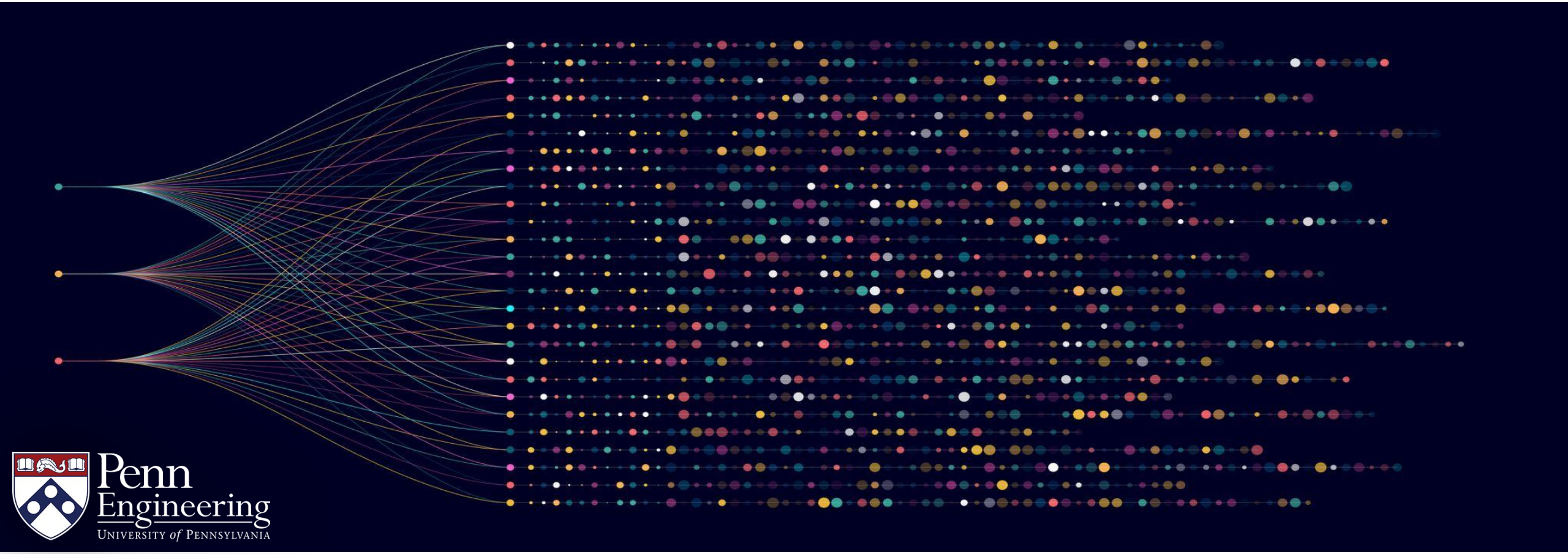
- Do there exist (more) efficient algorithms when allowing approximation and randomization?

- What is the tradeoff between the number of queries, the accuracy parameter, and failure probability?

Algorithms for Big Data

Anindya De Erik Waingarten

Description of the Algorithm



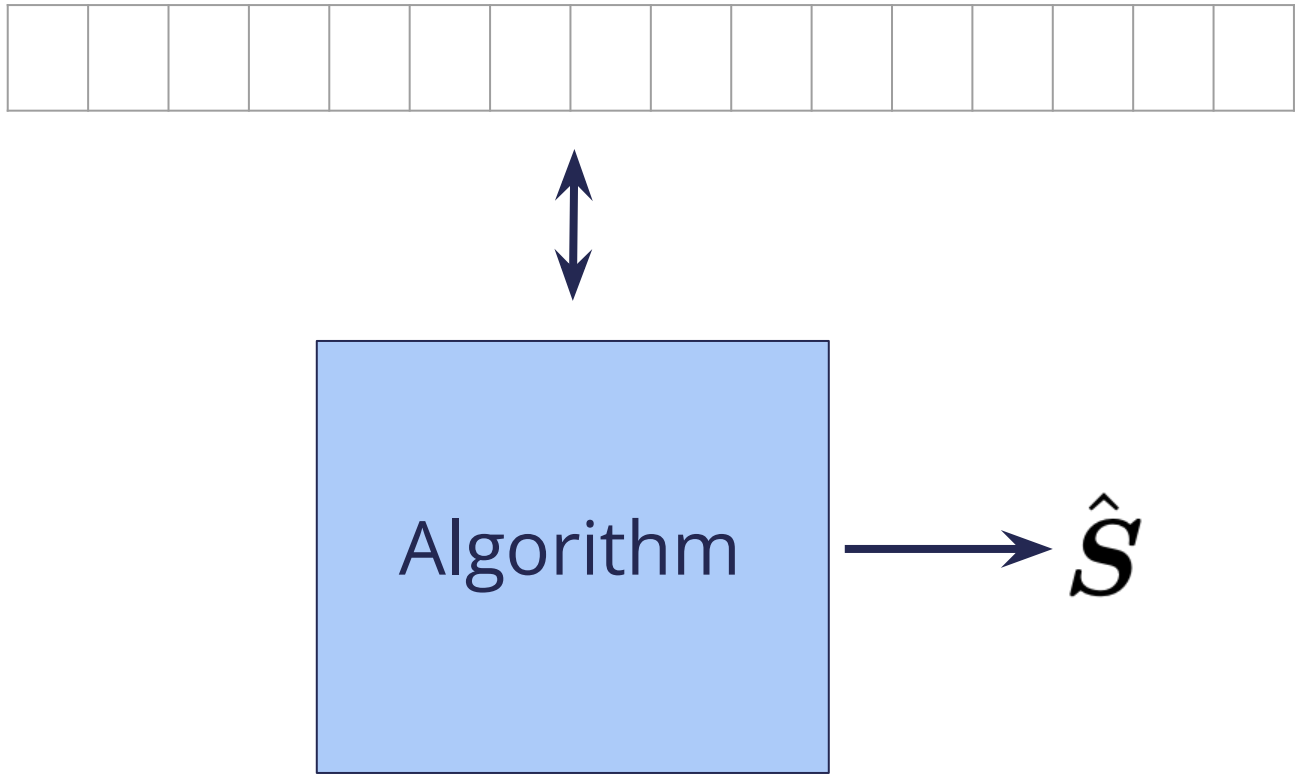
Penn Engineering
University of Pennsylvania

Objectives: Computing Large Averages

- Algorithm for approximating large averages

- Questions for the analysis

Algorithm Description



1. Sample q positions:

$$i_1, \ldots, i_q \sim [n]$$

1. Output average:

$$\hat{S} = \frac{1}{q} \sum_{t=1}^{q} X_{i_t}$$

Understanding the algorithm:

- Queries are each chosen uniformly and independently
- For every $j \in [n]$,

$$\Pr_{\boldsymbol{i}_t \sim [n]} [\boldsymbol{i}_t = j] = \frac{1}{n}$$

1. Sample q positions:

$$\boldsymbol{i}_1, \ldots, \boldsymbol{i}_q \sim [n]$$

1. Output average:

$$\hat{\boldsymbol{S}} = \frac{1}{q} \sum_{t=1}^{q} X_{\boldsymbol{i}_t}$$

Example Execution of Algorithm:

0	0	1	1	1	1	0	1	0	1	1	0	0	0	1	1

$$\boldsymbol{i}_1 =$$

$$\boldsymbol{i}_2 =$$

$$\vdots$$

$$\boldsymbol{i}_q =$$

source of randomness:

1. Sample q positions:

$$\boldsymbol{i}_1, \ldots, \boldsymbol{i}_q \sim [n]$$

1. Output average:

$$\hat{\boldsymbol{S}} = \frac{1}{q} \sum_{t=1}^{q} X_{\boldsymbol{i}_t}$$

Example Execution of Algorithm:

0	0	1	1	1	1	0	1	0	1	1	0	0	0	1	1

$$\boldsymbol{i}_1 = 11$$

$$\boldsymbol{i}_2 = 8$$

$$\vdots$$

$$\boldsymbol{i}_q = 15$$

source of randomness:

1. Sample q positions:

$$\boldsymbol{i}_1, \ldots, \boldsymbol{i}_q \sim [n]$$

1. Output average:

$$\hat{\boldsymbol{S}} = \frac{1}{q} \sum_{t=1}^{q} X_{\boldsymbol{i}_t}$$

Example Execution of Algorithm:

0	0	1	1	1	1	0	1	0	1	1	0	0	0	1	1

$$\boldsymbol{i}_1 = 11$$

$$\boldsymbol{i}_2 = 8$$

$$\vdots$$

$$\boldsymbol{i}_q = 15$$

source of randomness:

1. Sample q positions:

$$\boldsymbol{i}_1, \ldots, \boldsymbol{i}_q \sim [n]$$

1. Output average:

$$\hat{\boldsymbol{S}} = \frac{1}{q} \sum_{t=1}^{q} X_{\boldsymbol{i}_t}$$

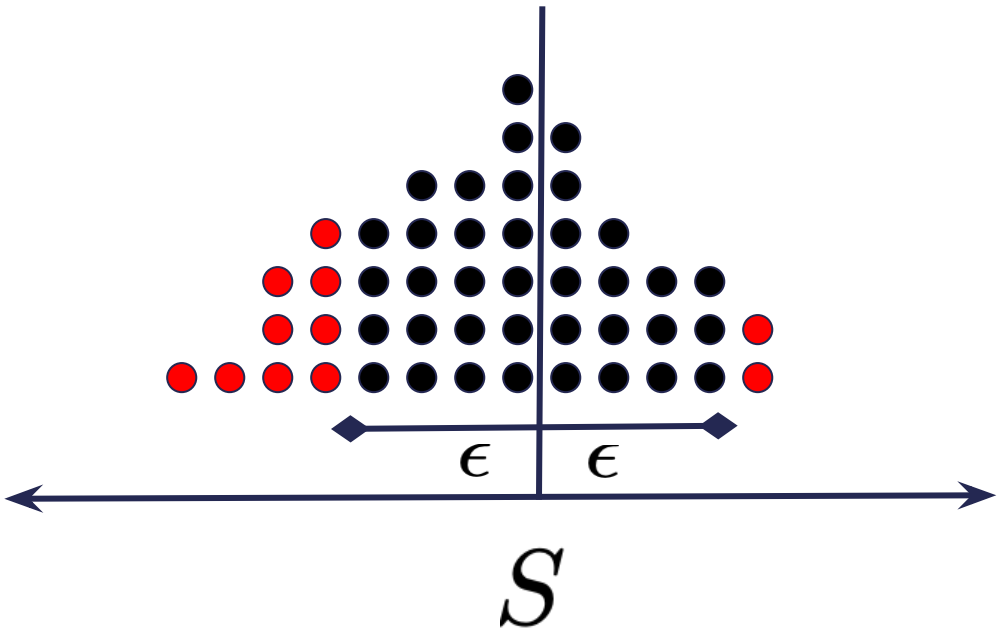
Questions for the Algorithm Analysis:

- How many queries do we need?

$$\Pr_{\boldsymbol{i}_1,\ldots,\boldsymbol{i}_q \sim [n]} \left[\left| \hat{\boldsymbol{S}} - S \right| \leq \epsilon \right] \geq 1 - \delta.$$

- Should depend on ϵ, δ.

Questions for the Algorithm Analysis:



$$\hat{S} = \frac{1}{q}\sum_{t=1}^{q} X_{i_t}$$

$$\Pr_{i_1,\ldots,i_q \sim [n]}\left[\left|\hat{S} - S\right| \leq \epsilon\right] \geq 1 - \delta.$$

ϵ : accuracy δ: failure probability

Toward the Analysis:

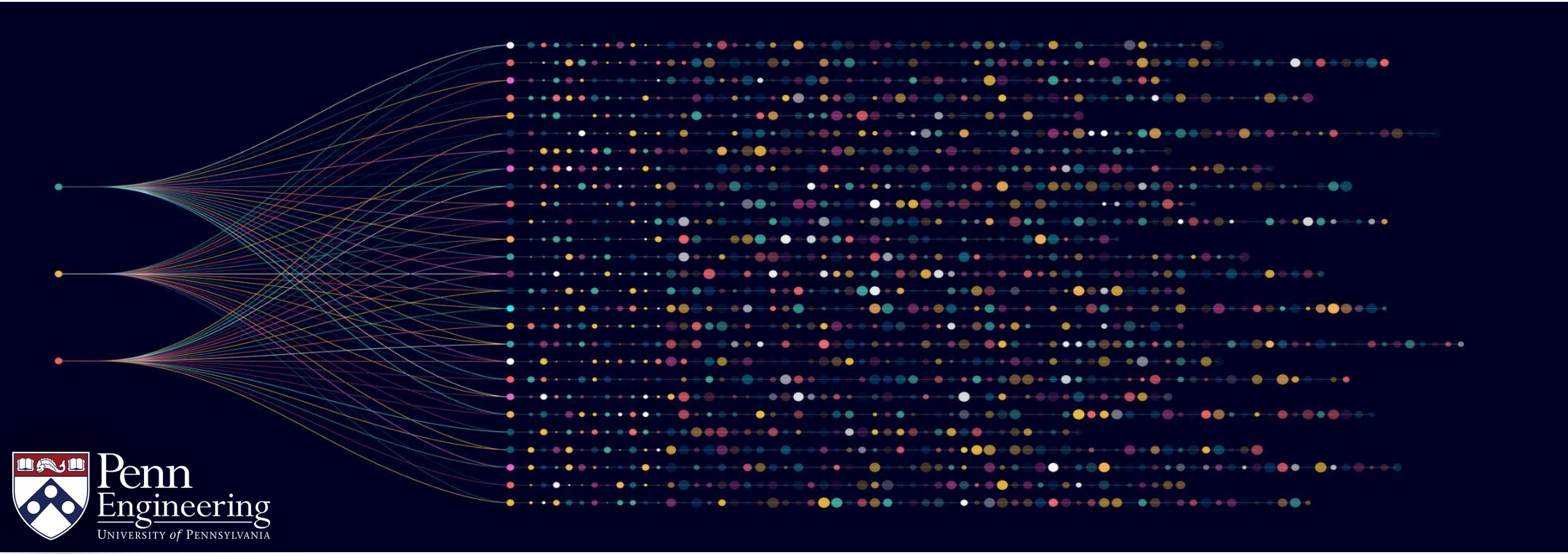
- If we took infinitely many queries, would the algorithm's output approach the average?
  - Hint: think of the expectation and the central-limit theorem!

- Suppose yes, with infinitely many queries. What happens with finitely many queries?
  - How close is an outcome to its expectation?
  - Hint: think of the variance and Chebyshev's inequality

Algorithms for Big Data

Anindya De Erik Waingarten

Analysis of the Algorithm



Penn Engineering
University of Pennsylvania

Objectives: Computing Large Averages

- Analysis of algorithm

- Expectation and Variance analysis

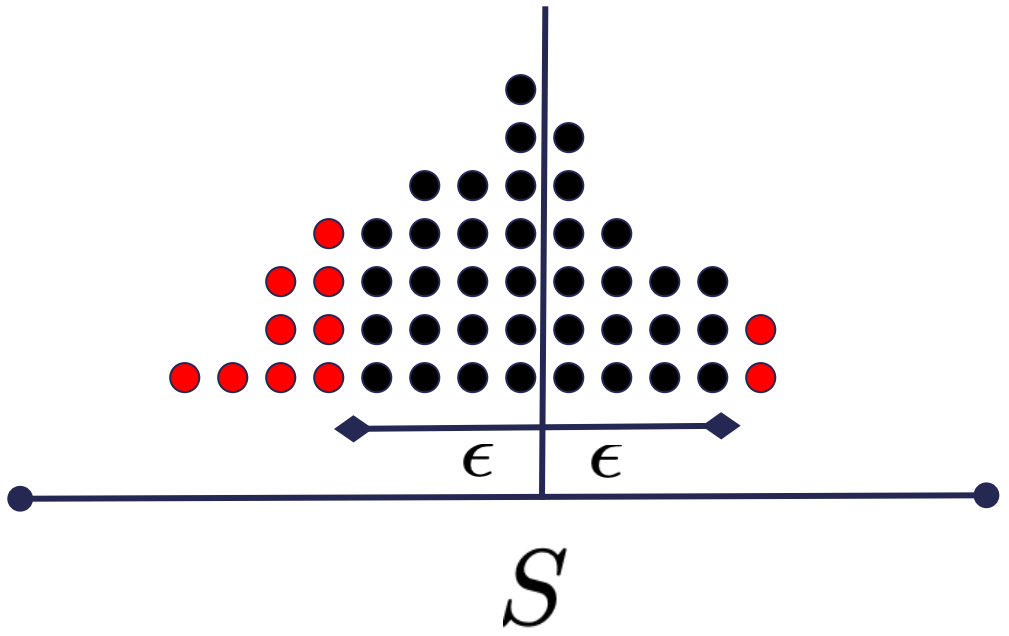
Analysis of the Algorithm

1. Sample q positions:

$$\boldsymbol{i}_1, \ldots, \boldsymbol{i}_q \sim [n]$$

2. Output empirical average:

$$\hat{\boldsymbol{S}} = \frac{1}{q} \sum_{t=1}^{q} X_{\boldsymbol{i}_t}$$



ϵ : accuracy δ : failure probability

Chebyshev's Inequality

- For any random variable $\boldsymbol{Z}$ with expectation $\mathbb{E}[\boldsymbol{Z}] = \mu$

$$\Pr\left[|\boldsymbol{Z} - \mu| \geq \epsilon\right] \leq \frac{\mathrm{Var}[\boldsymbol{Z}]}{\epsilon^2}$$

- Exactly what we need:

$$\Pr\left[\left|\hat{\boldsymbol{S}} - S\right| \geq \epsilon\right] \leq \frac{\mathrm{Var}[\hat{\boldsymbol{S}}]}{\epsilon^2}$$

The Plan:

- Compute the expectation of $\hat{S}$

- Compute an upper bound on $\mathrm{Var}[\hat{S}]$
  - Variance will depend on q

$$\Pr\left[\left|\hat{S} - S\right| \geq \epsilon\right] \leq \frac{\mathrm{Var}[\hat{S}]}{\epsilon^2}$$

Analysis via Chebyshev's Inequality

- Simplifying Expectation of $\hat{S}$:

$$\mathop{\mathbb{E}}_{i_1,\ldots,i_q \sim [n]} \left[\hat{S}\right] = \mathop{\mathbb{E}}_{i_1,\ldots,i_q \sim [n]} \left[\frac{1}{q}\sum_{t=1}^{q} X_{i_t}\right]$$

$$= \frac{1}{q}\sum_{t=1}^{q} \mathop{\mathbb{E}}_{i_t \sim [n]} \left[X_{i_t}\right]$$

Analysis via Chebyshev's Inequality

- Simplifying Expectation of $\hat{S}$:

$$\mathop{\mathbb{E}}_{i_1,\ldots,i_q \sim [n]} \left[\hat{S}\right] = \frac{1}{q} \sum_{t=1}^{q} \mathop{\mathbb{E}}_{i_t \sim [n]} \left[X_{i_t}\right]$$

- Expectation of each term:

$$\mathop{\mathbb{E}}_{i_t \sim [n]} \left[X_{i_t}\right] = \sum_{j=1}^{n} \Pr[i_t = j] \cdot X_j = \frac{1}{n} \sum_{j=1}^{n} X_j = S.$$

Chebyshev's Inequality

- For any random variable $\boldsymbol{Z}$ with expectation $\mathbb{E}[\boldsymbol{Z}] = \mu$

$$\Pr\left[|\boldsymbol{Z} - \mu| \geq \epsilon\right] \leq \frac{\mathrm{Var}[\boldsymbol{Z}]}{\epsilon^2}$$

- Exactly what we need:

$$\Pr\left[\left|\hat{\boldsymbol{S}} - S\right| \geq \epsilon\right] \leq \frac{\mathrm{Var}[\hat{\boldsymbol{S}}]}{\epsilon^2}$$

Task:

- Compute an upper bound on $\mathrm{Var}[\hat{S}]$ as a function of q
  - As q increases, variance will decrease

- Set q large enough so that

$$\Pr\left[\left|\hat{S} - S\right| \geq \epsilon\right] \leq \frac{\mathrm{Var}[\hat{S}]}{\epsilon^2} \leq \delta.$$

Variance:

- Simplifying Variance:

$$\mathrm{Var}[\hat{\boldsymbol{S}}] = \mathrm{Var}\left[\frac{1}{q}\sum_{t=1}^{q} X_{\boldsymbol{i}_t}\right]$$

$$= \frac{1}{q^2}\sum_{t=1}^{q} \mathrm{Var}\left[X_{\boldsymbol{i}_t}\right]$$

Variance:

- Simplifying Variance:

$$\mathrm{Var}[\hat{\boldsymbol{S}}] = \frac{1}{q^2} \sum_{t=1}^{q} \mathrm{Var}\left[X_{\boldsymbol{i}_t}\right]$$

- Variance of single term:

$$\begin{aligned} \mathrm{Var}\left[X_{\boldsymbol{i}_t}\right] &= \mathbb{E}\left[\left(X_{\boldsymbol{i}_t} - S\right)^2\right] \\ &\leq 1. \end{aligned}$$

Variance:

- Simplifying Variance:

$$\mathrm{Var}[\hat{\boldsymbol{S}}] = \frac{1}{q^2}\sum_{t=1}^{q}\mathrm{Var}\left[X_{\boldsymbol{i}_t}\right] \leq \frac{1}{q}$$

- Variance of single term:

$$\begin{aligned}\mathrm{Var}\left[X_{\boldsymbol{i}_t}\right] &= \mathbb{E}\left[\left(X_{\boldsymbol{i}_t} - S\right)^2\right] \\ &\leq 1.\end{aligned}$$

Chebyshev's inequality:

- Substituting bound on variance:

$$\Pr\left[\left|\hat{\boldsymbol{S}} - S\right| \geq \epsilon\right] \leq \frac{\mathrm{Var}[\hat{\boldsymbol{S}}]}{\epsilon^2} \leq \frac{1}{q\epsilon^2}.$$

- Set $q = \dfrac{1}{\delta\epsilon^2}$:

$$\Pr\left[\left|\hat{\boldsymbol{S}} - S\right| \geq \epsilon\right] \leq \delta$$

Complete Description of Algorithm:

1. Sample $q = \frac{1}{\delta \epsilon^2}$ positions:

$$\boldsymbol{i}_1, \ldots, \boldsymbol{i}_q \sim [n]$$

2. Output empirical average:

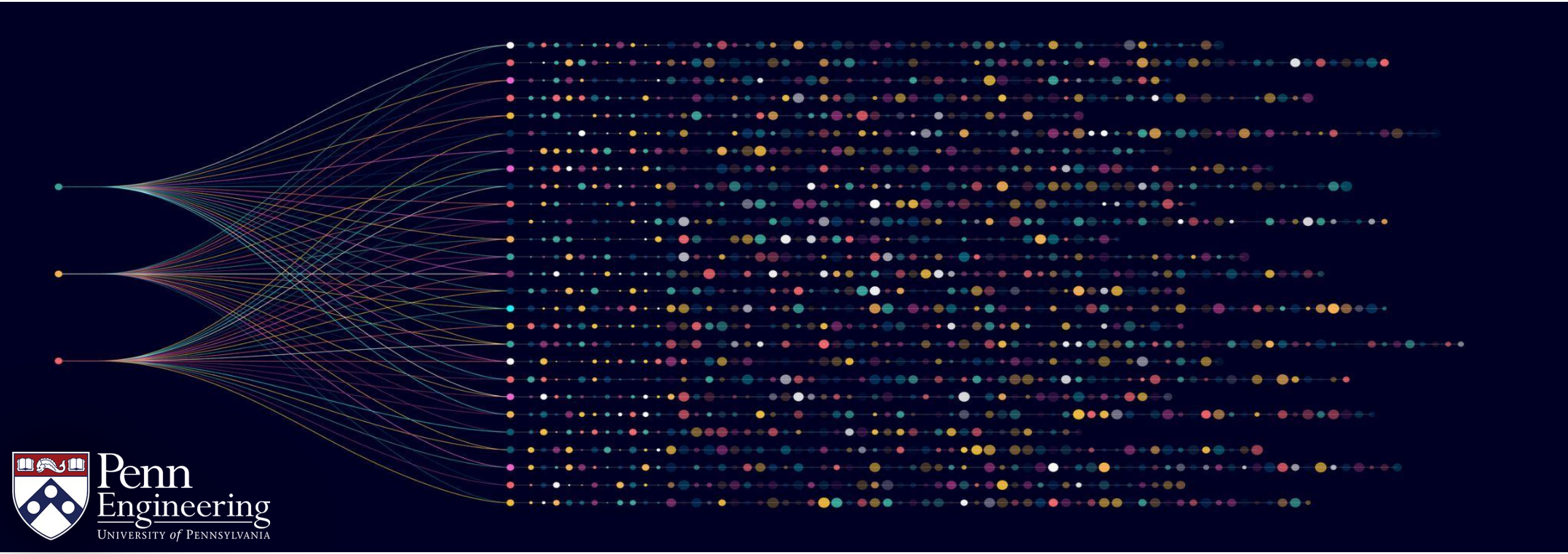
$$\hat{\boldsymbol{S}} = \frac{1}{q} \sum_{t=1}^{q} X_{\boldsymbol{i}_t}$$

- Completely independent of the size of the data!
- Is it optimal?

Algorithms for Big Data

Anindya De Erik Waingarten

Example Setting of Parameters



Penn Engineering
University of Pennsylvania

Objectives: Computing Large Averages

- Example setting of parameters

- Optimality with respect to δ

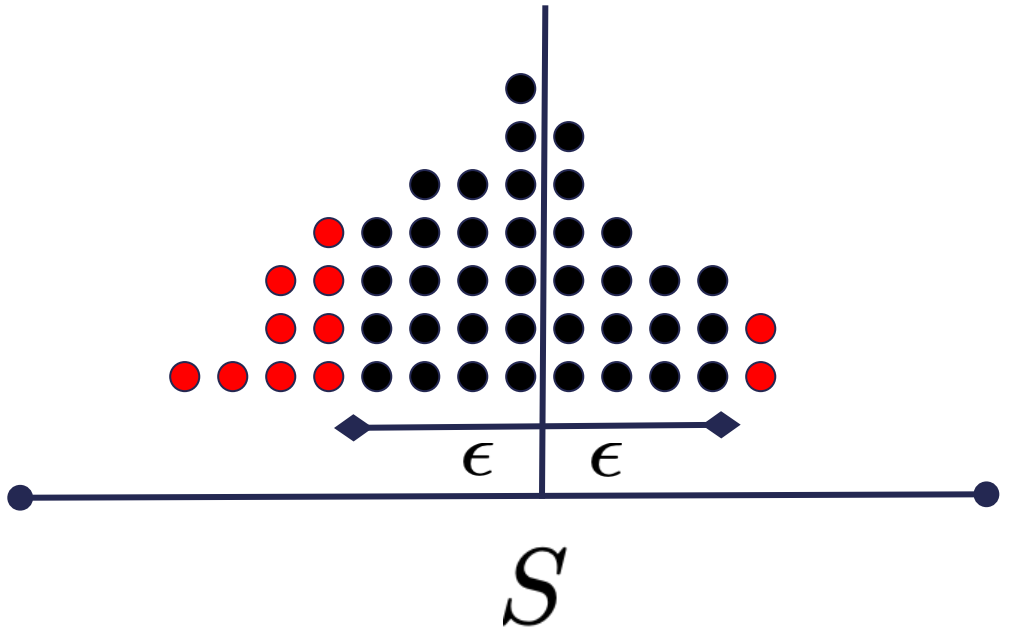
Analysis of the Algorithm

1. Sample $q = \dfrac{1}{\delta\epsilon^2}$ positions:

$$\boldsymbol{i}_1, \ldots, \boldsymbol{i}_q \sim [n]$$

2. Output empirical average:

$$\hat{\boldsymbol{S}} = \frac{1}{q}\sum_{t=1}^{q} X_{\boldsymbol{i}_t}$$



$$\Pr\left[\left|\hat{\boldsymbol{S}} - S\right| \geq \epsilon\right] \leq \delta$$

How many users are in Europe?

- Total number of Facebook users: ~3 billion

- For accuracy parameter 0.01, and failure probability 0.1, it suffices to consider 100,000 queries.
  - That's only 0.003% of the dataset!

- The output is correct 90% of the time. If it is correct, the fraction of European users is $\hat{S}$ up to plus or minus 0.01.

How many users are in Europe?

- Total number of Facebook users: ~3 billion

- For accuracy parameter ~~0.01~~ 0.05, and failure probability 0.1, it suffices to consider ~~100,000~~ 4,000 queries.

- The output is correct 90% of the time. If it is correct, the fraction of European users is $\hat{S}$ up to plus or minus ~~0.01~~ 0.05

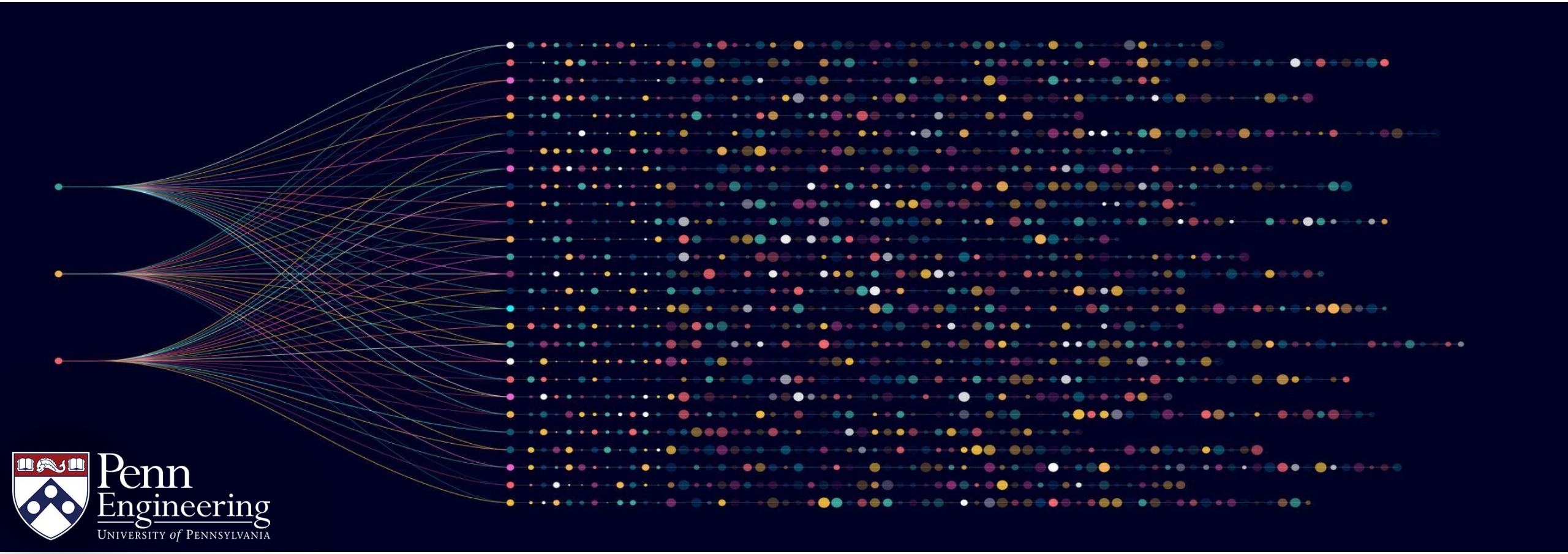
Is it optimal?

- Two parameters: accuracy and failure probability
- Query complexity: $q = \frac{1}{\delta\epsilon^2}$
- Dependence on ϵ is optimal
- Dependence on δ can be improved $O\left(\frac{\log(1/\delta)}{\epsilon^2}\right)$

Algorithms for Big Data

Anindya De Erik Waingarten

High Probability Guarantees



Penn Engineering
University of Pennsylvania

Objectives: Computing Large Averages

- Application of Chernoff/Hoeffding inequality

Chebyshev vs. Chernoff/Hoeffding

- Chebyshev Inequality: for random variable Z with $\mathbb{E}[Z] = \mu$

$$\Pr[|Z - \mu| \geq \epsilon] \leq \frac{\mathrm{Var}[Z]}{\epsilon^2}$$

- Chernoff/Hoeffding Inequality
  - If Z is a sum of q independent random variables between a and b:

$$\Pr[|Z - \mu| \geq \epsilon] \leq 2e^{-2\epsilon^2/(q(b-a)^2)}$$

Application of Chernoff/Hoeffding

- Chernoff/Hoeffding Inequality:

$$\Pr\left[\left|\boldsymbol{Z} - \mu\right| \geq \epsilon\right] \leq 2e^{-2\epsilon^2/(q(b-a)^2)}$$

- Our setting: $\hat{\boldsymbol{S}} = \frac{1}{q}\sum_{t=1}^{q} X_{\boldsymbol{i}_t} \qquad b = 1/q \qquad a = 0$

$$\Pr\left[\left|\hat{\boldsymbol{S}} - S\right| \geq \epsilon\right] \leq 2e^{-2\epsilon^2/(q(1/q)^2)} = 2e^{-2\epsilon^2 q}$$

- Setting $q = \frac{\log(2/\delta)}{2\epsilon^2}$ gives desired bound!

Complete Description of Algorithm:

1. Sample $q = \dfrac{\log(2/\delta)}{2\epsilon^2}$ positions:

$$\boldsymbol{i}_1, \ldots, \boldsymbol{i}_q \sim [n]$$

2. Output empirical average:

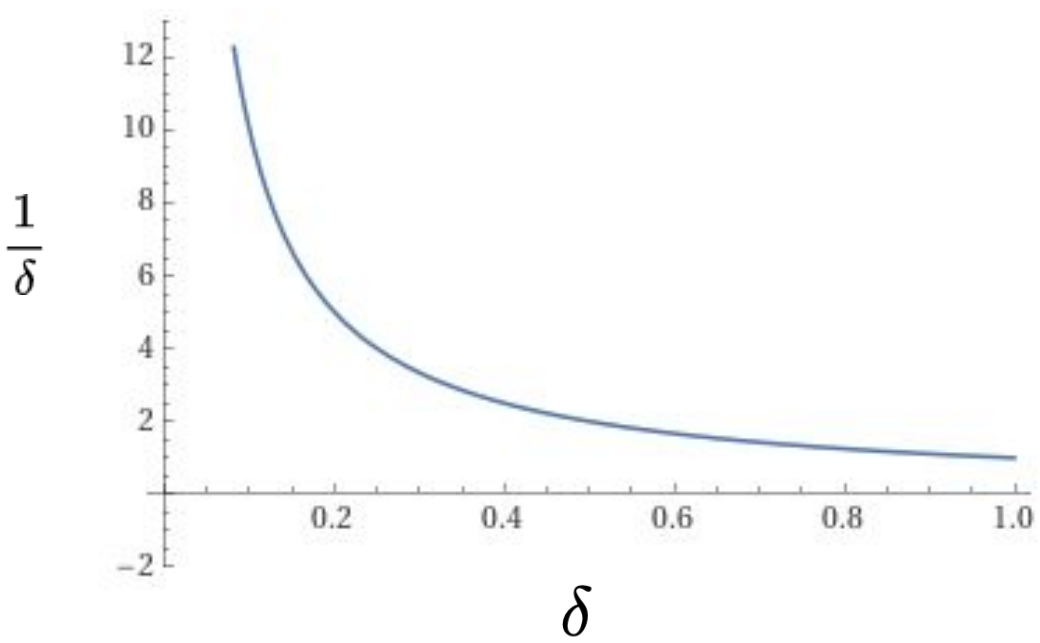
$$\hat{\boldsymbol{S}} = \frac{1}{q} \sum_{t=1}^{q} X_{\boldsymbol{i}_t}$$

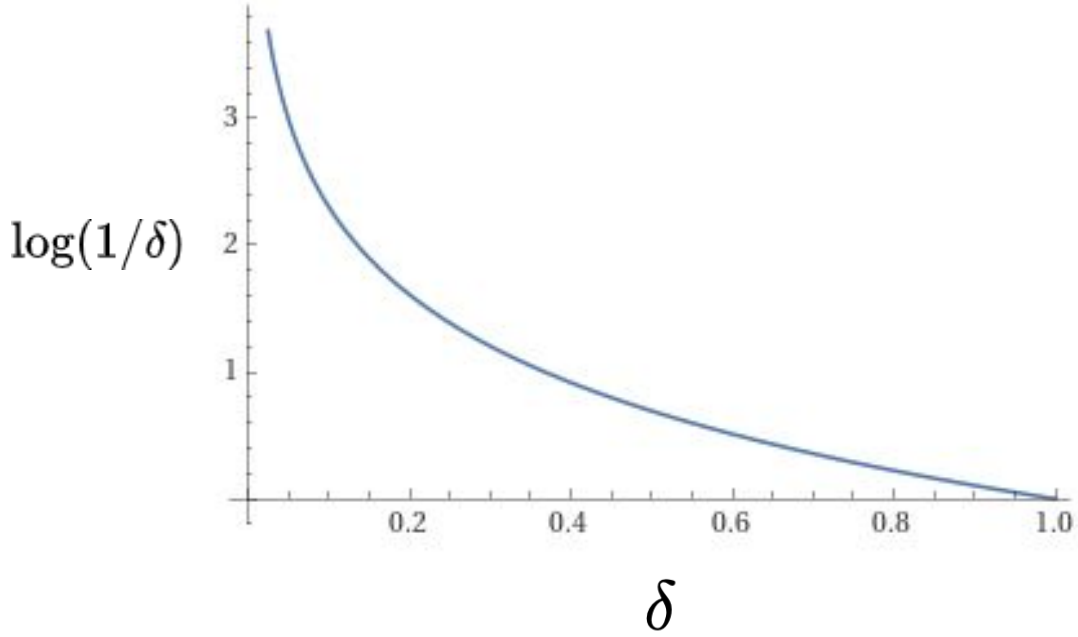
- Asymptotically optimal dependence on accuracy and failure probability

$$O\left(\frac{1}{\delta\epsilon^2}\right) \quad \text{vs.}$$

$$O\left(\frac{\log(1/\delta)}{\epsilon^2}\right)$$

Inverse vs logarithmic dependence





- Logarithmic dependence is always better:
  - For failure probability 0.01 – inverse 100, log (base 2) is 4.6

- Much more pronounced when failure probability is really tiny

Take-aways:

- Two parameters: accuracy and failure probability

$$q = O\left(\frac{\log(1/\delta)}{\epsilon^2}\right)$$

- Easy to make failure probability really *tiny*, but hard to make accuracy really *tiny*

ϵ

δ	0.1	0.01	0.001
0.1	231	23K	2.3M
0.01	460	46K	4.6M
0.001	690	69K	6.9M

Why should you care?

- Relatively cheap to make randomized algorithm succeed with very high probability

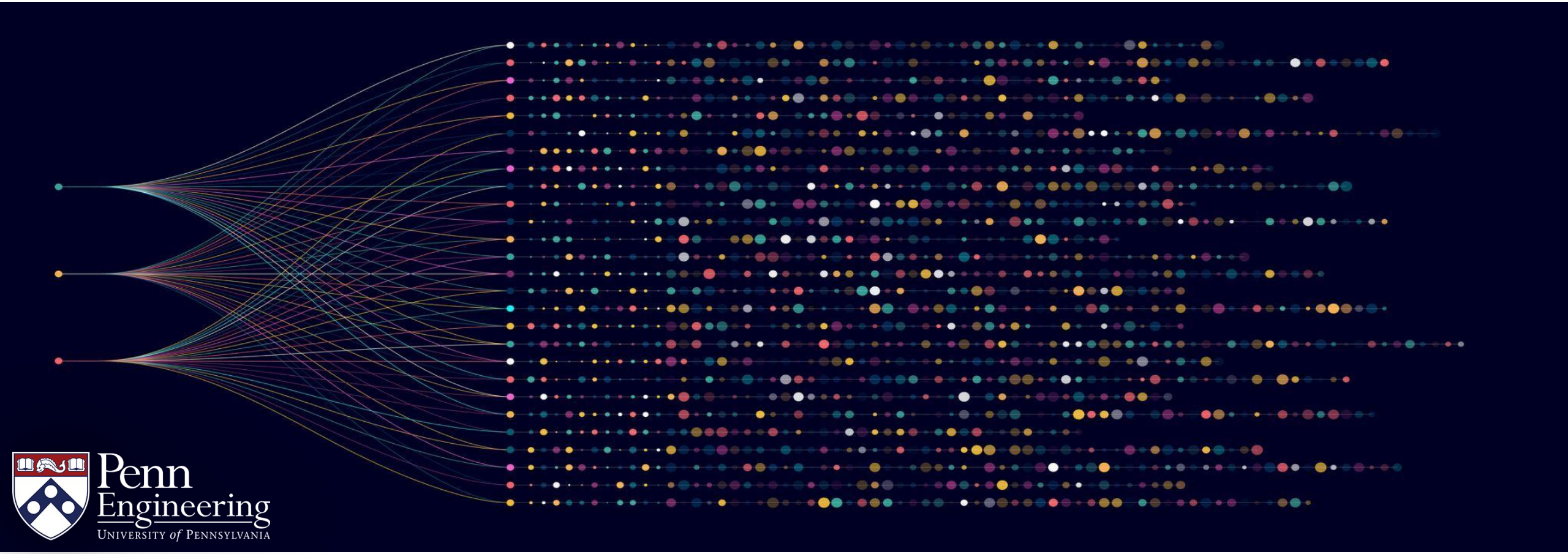
- If algorithm fails with probability at most 2^{-300}, it will essentially never fail
  - with inverse relationship: intractable.
  - with logarithmic relationship: tractable.

- Very important when using repeated estimations

Algorithms for Big Data

Anindya De Erik Waingarten

Small Summaries from Sampling



Penn Engineering
University of Pennsylvania

Objectives: Computing Large Averages

- Application: small summaries via random sampling

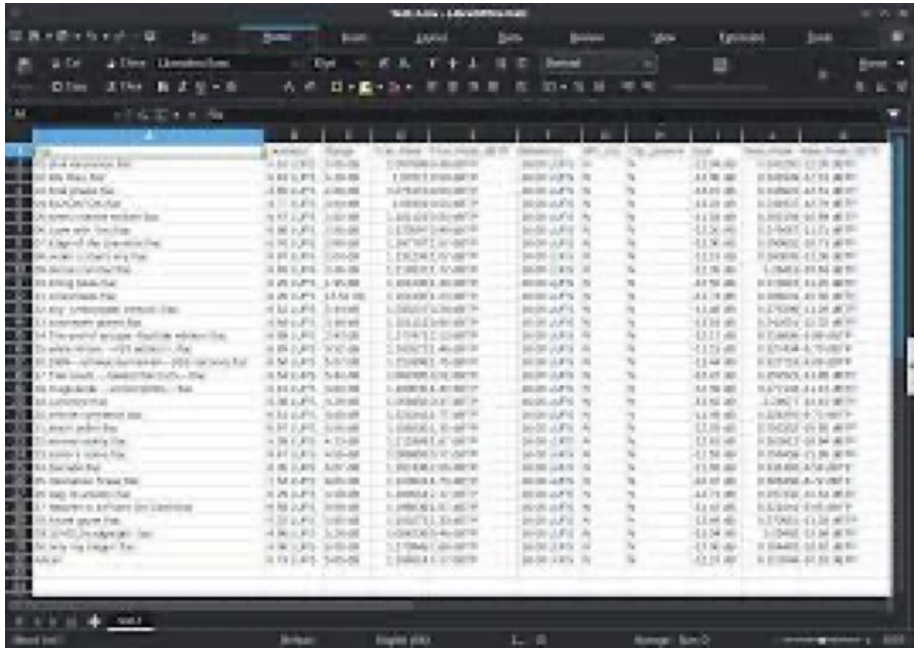
- Small failure probability critically important

Motivating Scenario:

- Users have many attributes
  - approx. statistics on a lot of different user behavior

- How many European? How many are German? British? How many posted today? Yesterday? Logged in twice in a week? ...

- Unclear which questions will arise

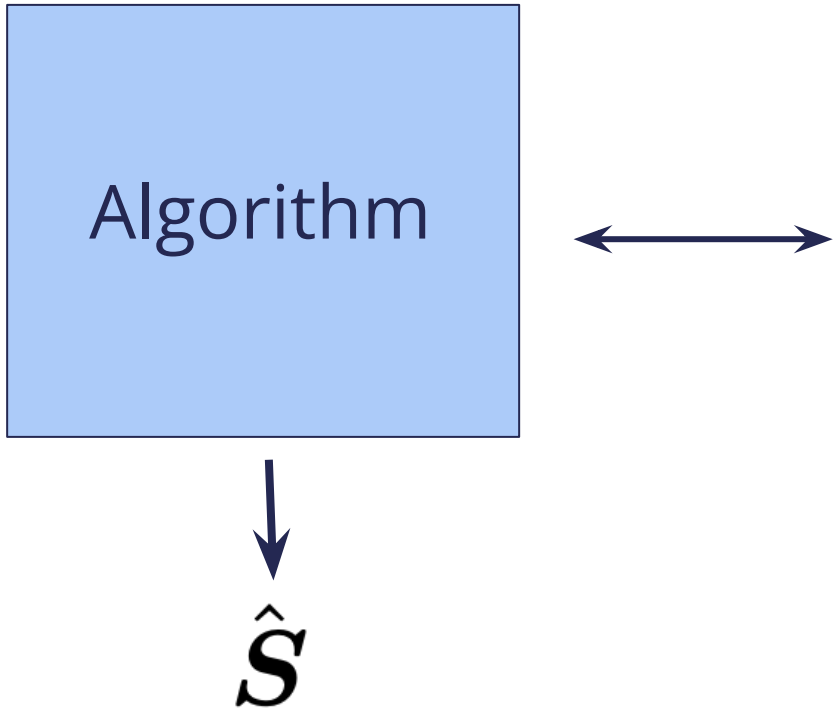


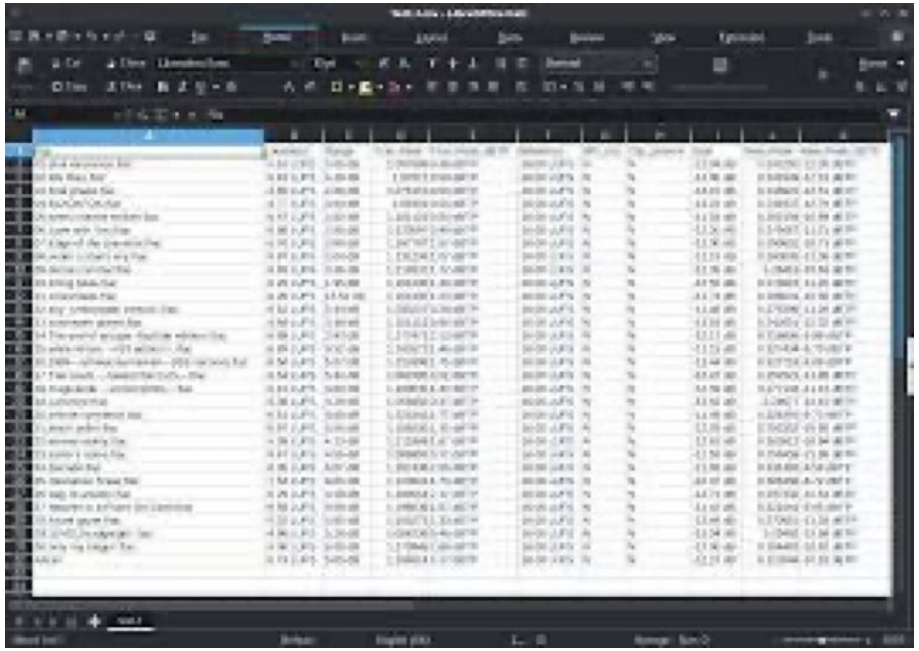
Rows are users,

columns are attributes

Motivating Scenario:

For each question:





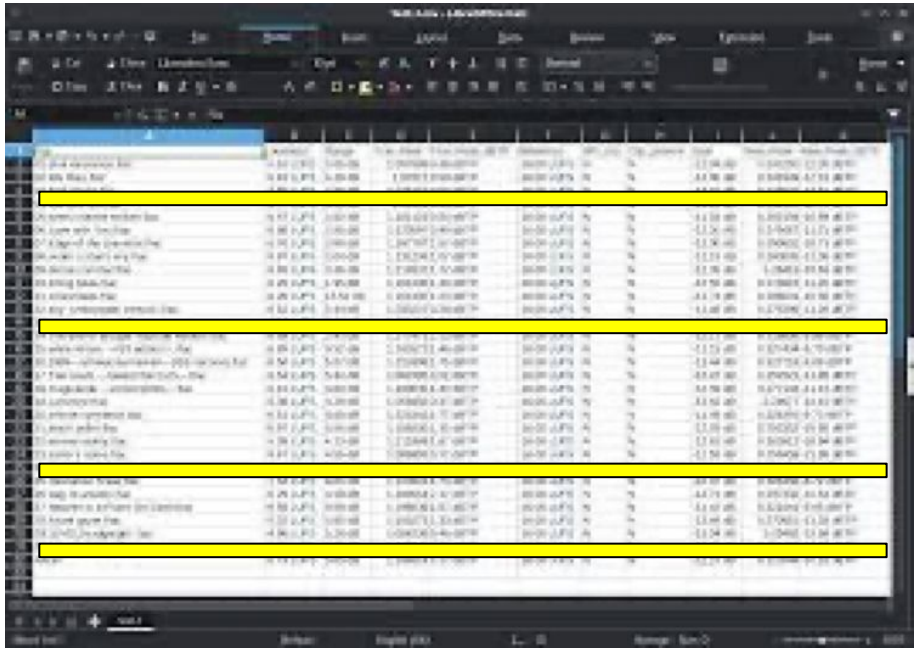
Rows are users,

columns are attributes

Motivating Scenario:

- Each execution generates a random sample of users

- If repeated for M questions, would be using M times q queries

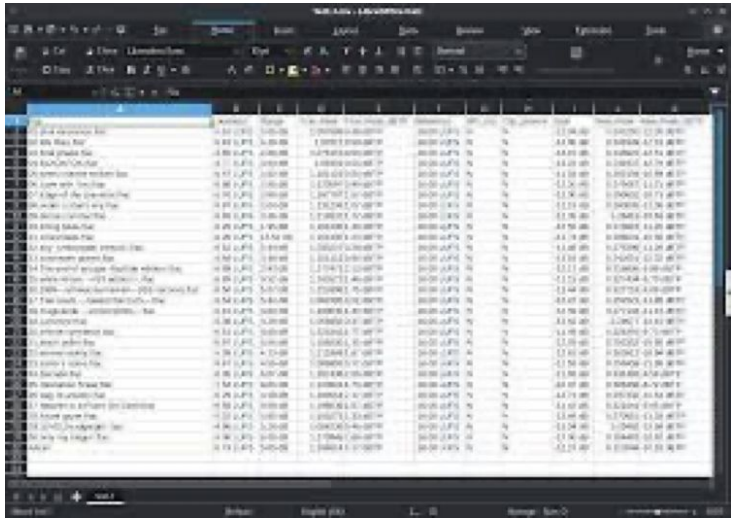


Rows are users,

columns are attributes

Motivating Scenario: Is it possible to…?

- Generate a *single* set of samples



- For each question, use the *same* set of samples to execute the algorithm

$$\boldsymbol{i}_1, \ldots, \boldsymbol{i}_q \sim [n]$$

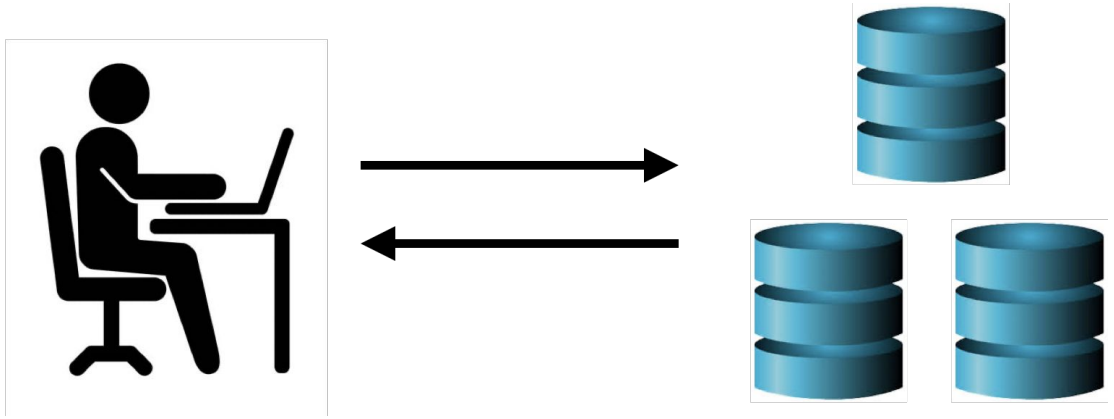
q

Benefits:

- Ideally, fewer samples than one batch per question
- Samples can be pre-generated and used multiple times
- Fewer interactions with central server (also be preemptively done)

Mathematical Formalism:

- Domain $\mathcal{X}$ of possible objects
  - ex. possible space of users
- There is a dataset $P \subset \mathcal{X}$ of n objects
  - ex. possible users
- Set of functions $\mathcal{F}$ mapping domain to [0,1]



$$f \in \mathcal{F} \quad : \quad f : \mathcal{X} \rightarrow [0, 1]$$

Example: Country of Origin for Users

- All possible users $\mathcal{X}$.
- Our dataset of users $P \subset \mathcal{X}$.
- Collection of 195 functions:

$$\mathcal{F} = \{f_{\text{USA}}, f_{\text{Canada}}, f_{\text{Egypt}}, \dots\}$$

$$f_{\text{USA}}(x) = \begin{cases} 1 & x \text{ from USA} \\ 0 & \text{otherwise} \end{cases}$$

Main Result:

Theorem: For any collection of functions $\mathcal{F}$, any dataset P, and any accuracy parameter ϵ and failure probability δ.

Letting

$$q = \frac{\log(2|\mathcal{F}|/\delta)}{2\epsilon^2}$$

A summary of q random samples can ϵ-approximate every function $f \in \mathcal{F}$ with probability at least $1 - \delta$

Applying the Theorem:

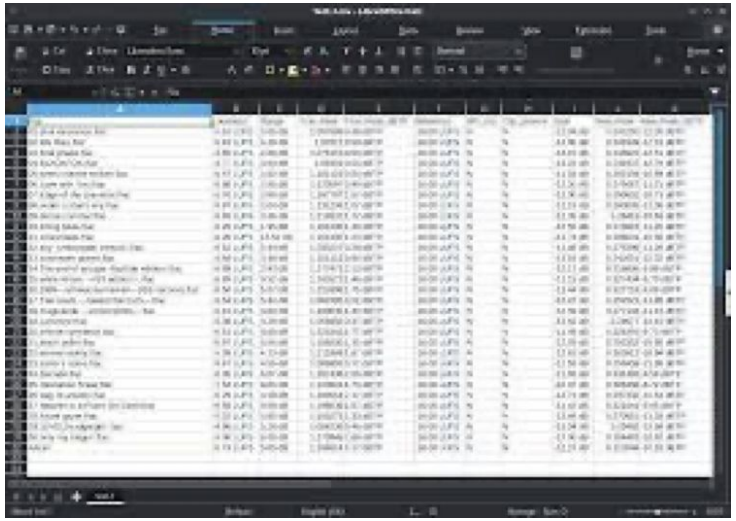
- Fix collection of functions $\mathcal{F}$

- Estimate statistic:

$$\frac{1}{q}\sum_{t=1}^{q} f(x_{i_t})$$

- Number of samples:

$$O(\log(|\mathcal{F}|/\delta)/\epsilon^2)$$



$$i_1, \ldots, i_q \sim [n]$$

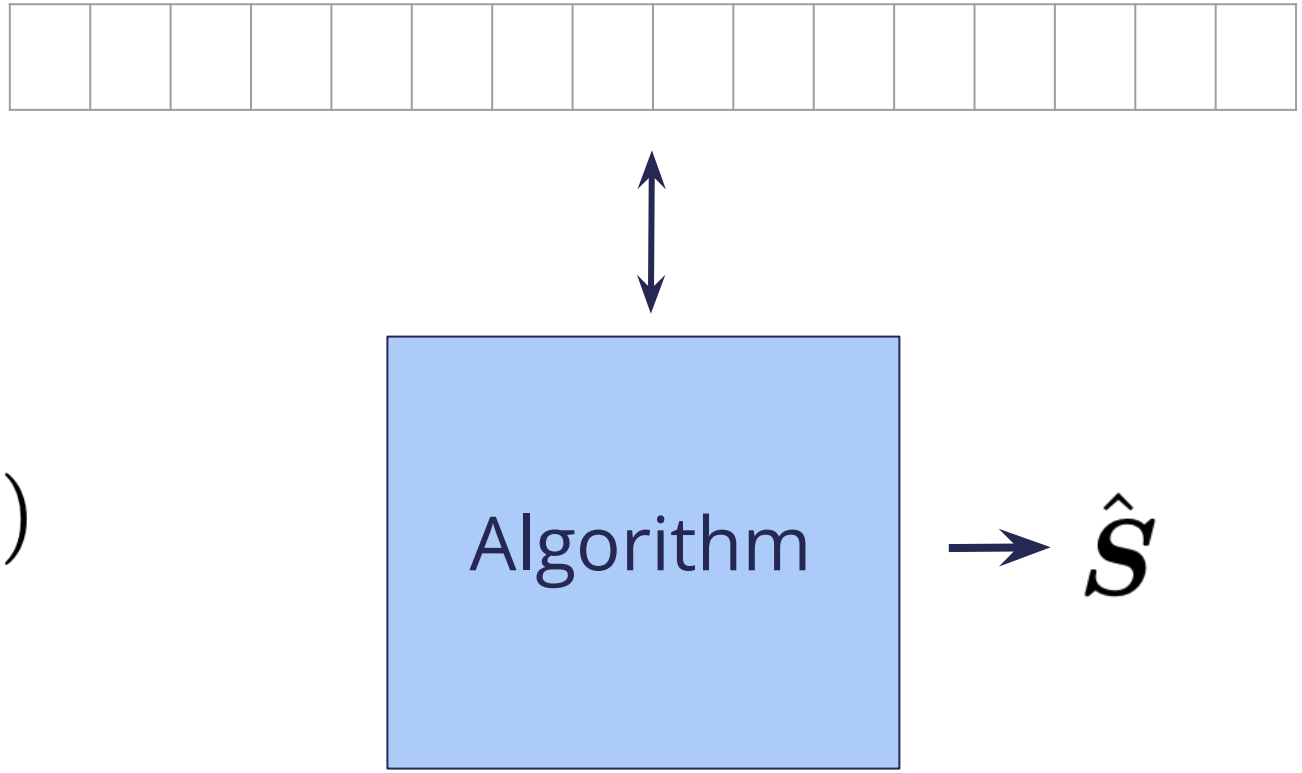
q

Analysis: Union bound over collection.

- Consider single statistic

$$f : \mathcal{X} \to [0, 1]$$

$$\frac{1}{n}\sum_{t=1}^{n} f(x_i) \stackrel{?}{=} \frac{1}{q}\sum_{t=1}^{q} f(x_{i_t})$$



- Failure probability:

$$\Pr\left[|S - \hat{S}| \geq \epsilon\right] \leq 2e^{-2\epsilon^2 q}$$

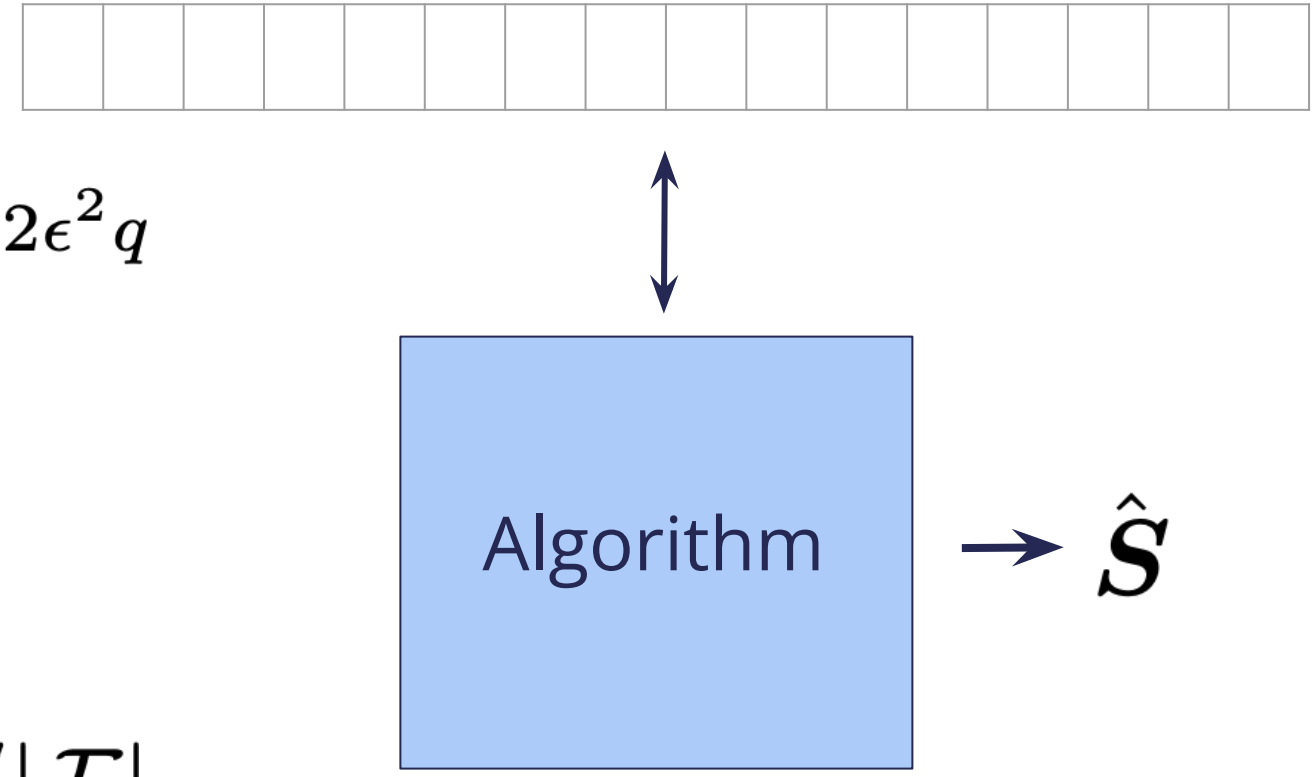
Analysis: Union bound over collection.

- Failure probability:

$$\Pr\left[|S - \hat{\boldsymbol{S}}| \geq \epsilon\right] \leq 2e^{-2\epsilon^2 q}$$

- When $q = \dfrac{\log(2|\mathcal{F}|/\delta)}{2\epsilon^2}$,

$$\leq \delta/|\mathcal{F}|$$

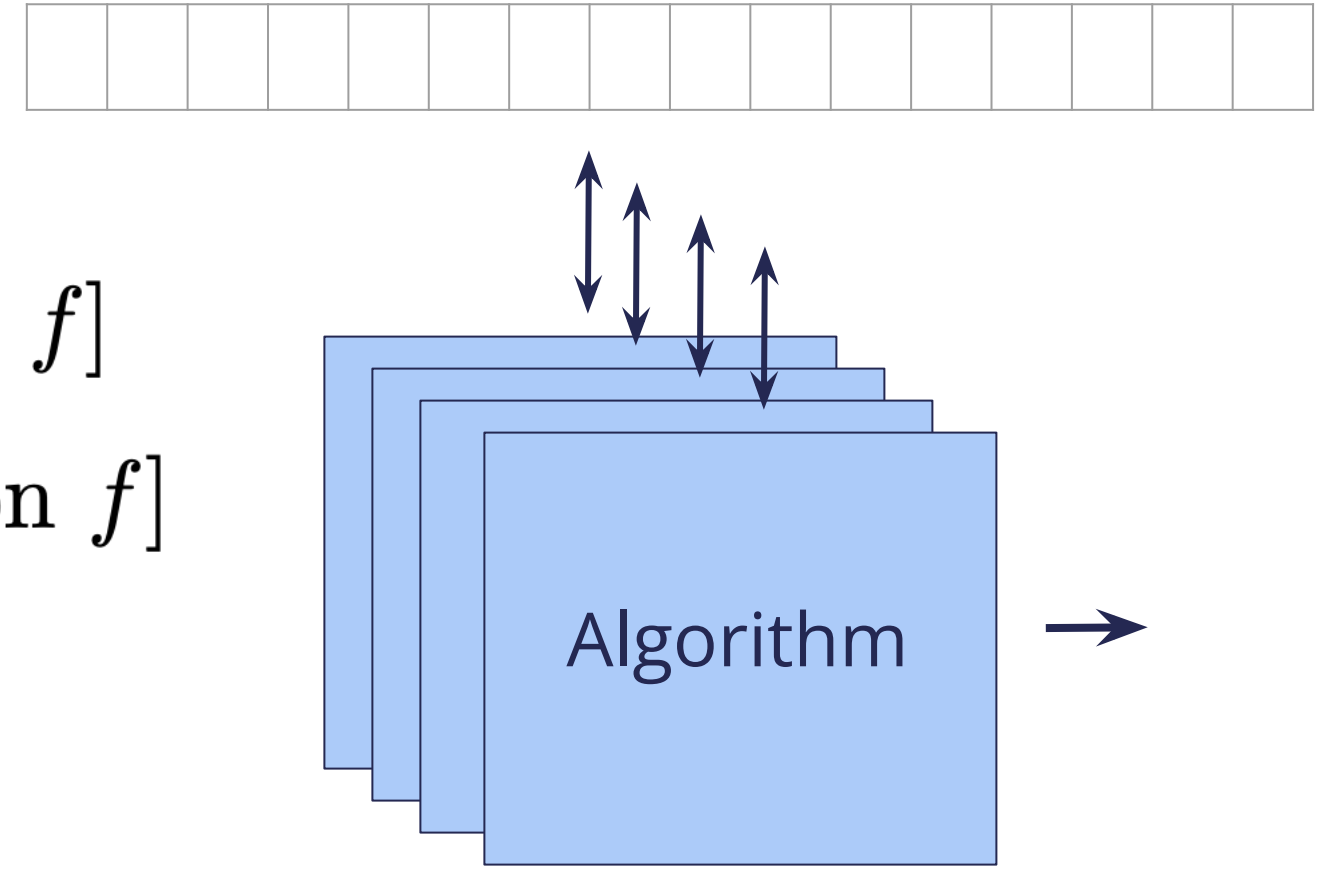


Analysis: Union bound over collection.

- Probability that there exists a mistake:

$$\Pr\left[\exists f \in \mathcal{F} : \text{ mistake on } f\right]$$
$$\leq \sum_{f \in \mathcal{F}} \Pr\left[\text{mistake on } f\right]$$
$$\leq |\mathcal{F}| \cdot \delta / |\mathcal{F}|$$

by union bound



Summary

- Random sampling gives small summaries (“sketches”) of big data
  - For fixed collection of statistics, dramatic data reduction

δ \ ϵ	0.1	0.01	0.001
0.1	413	41K	4.1M
0.01	529	53K	5.3M
0.001	644	64K	6.4M

- Important caveat: functions must be fixed in advanced!